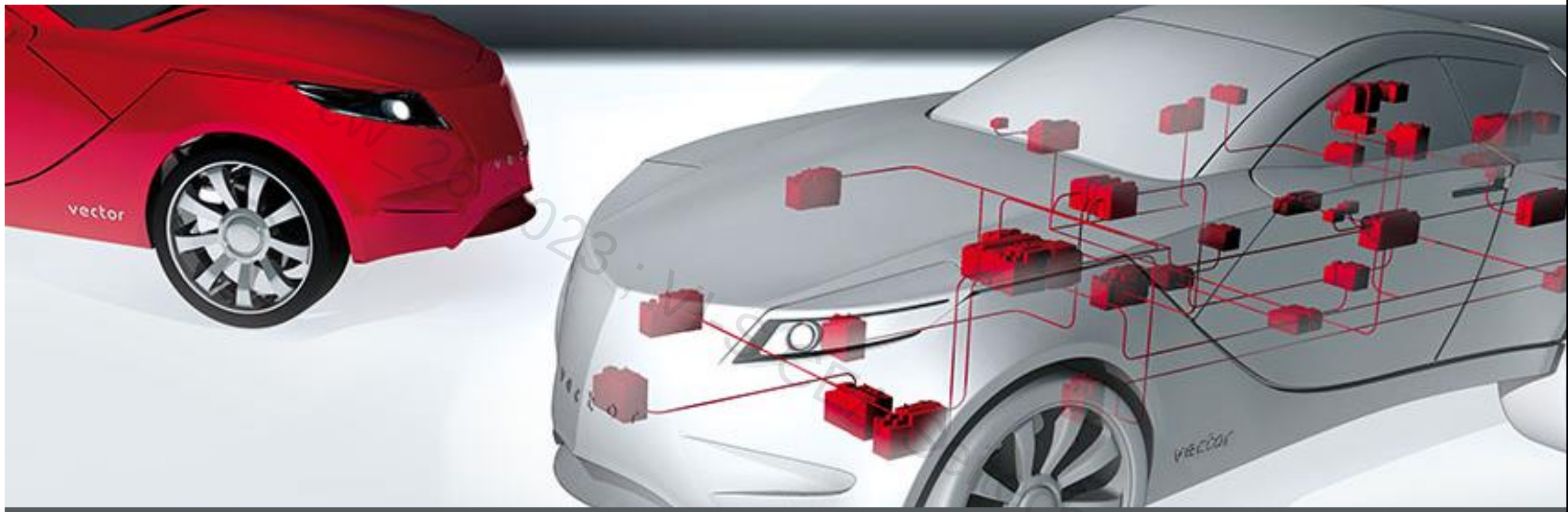
VECTOR



AUTOSAR in Practice

Mode Management

Agenda

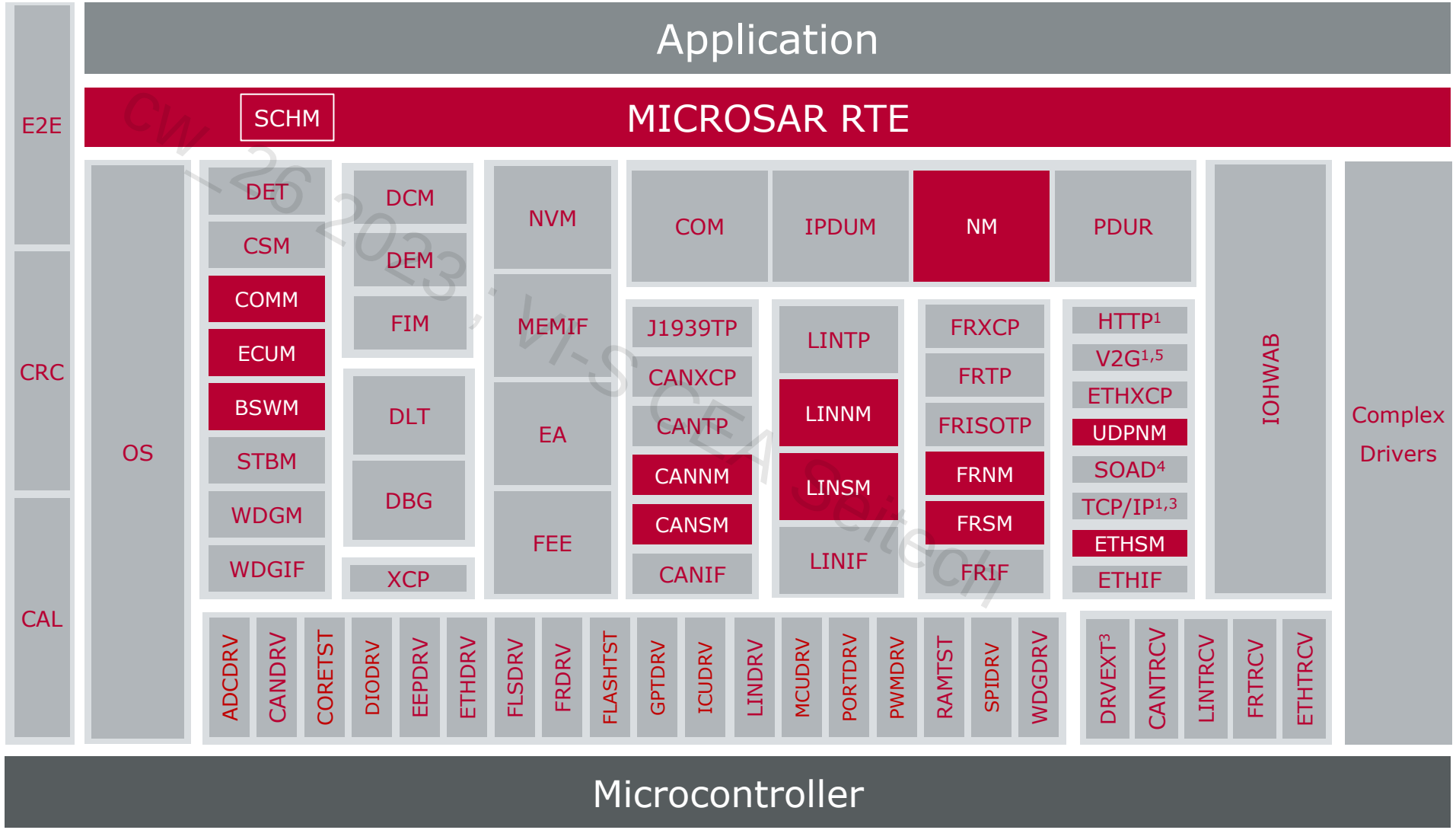
- **Mode Management**
- Wakeup Handling
- BSWM configuration
- ECUM configuration
- Mode Management
- Network Management Algorithm
- Mode Manager Concept
- Service Mapping
- Coming up next

cw_26 2023 ; VI-S CEA Seitech

VECTOR

Affected BSW modules

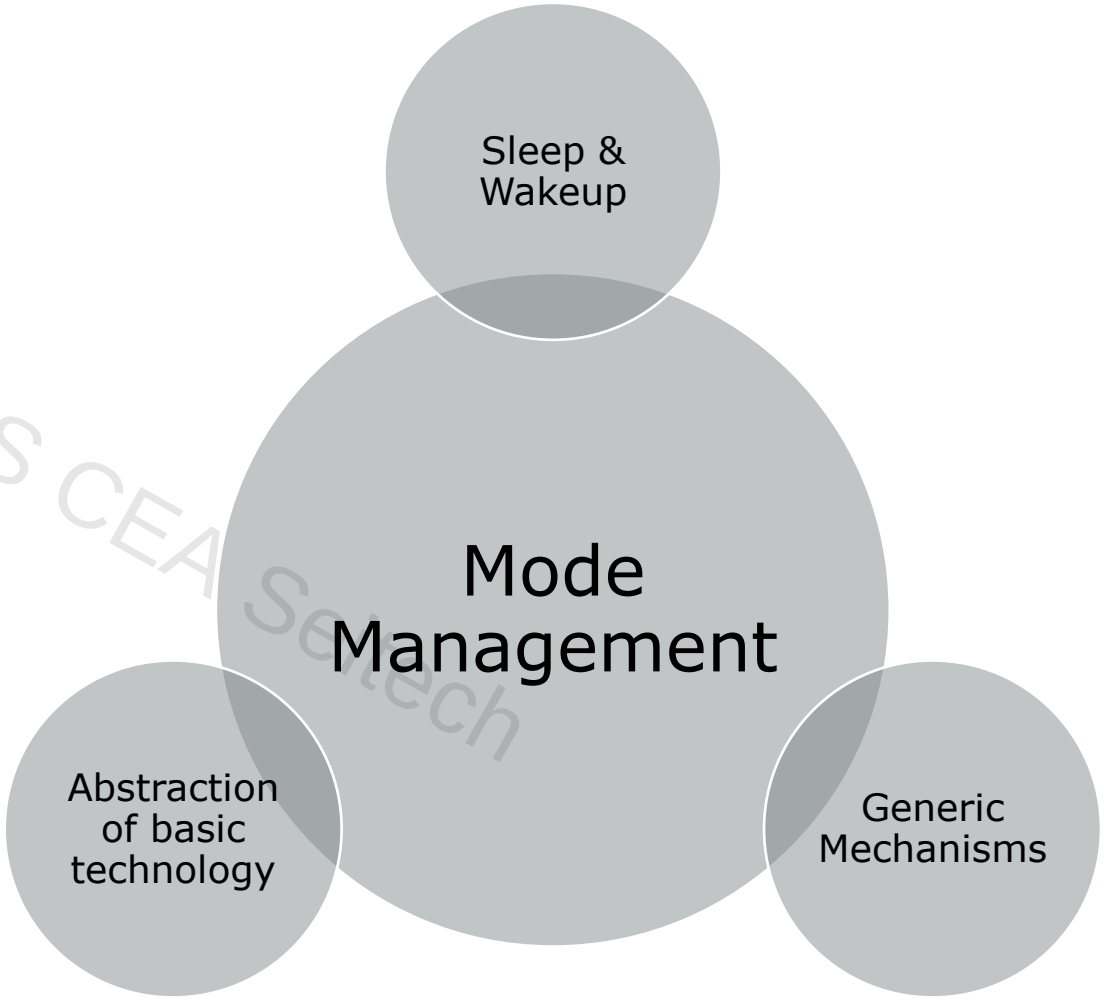
- ECUM
- BSWM
- COMM
- NM
- CANNM (FRNM, LINNM, UDPNM)
- CANSM (LINSM, FRSM, ETHSM)



CW 26 2023 - VI-SCEA Seitech

Motivation

1. The BSW shall satisfy specific requirements for an ECU
   - "Wakeup and Sleep" functionality in the following cases, for example:
     - ECU operation (µC and local peripherals)
     - Communication channel handling (vehicle networks)
2. Abstraction of technology from the application (same treatment of different basic technologies like CAN, LIN, FR, ETH)
3. Generic concept for (both user-defined and BSW-defined) mode management
   - Mechanisms for notification
   - Service requests for mode changes (e.g., by SWCs)
   - Mode arbitration (decision on how to handle transitions)
   - Mode control (execution of mode transitions)



CW 38 2023 / VI-S CEA Setech

ECU State Manager

ECUM

- AUTOSAR 4.x defines
  - ECUM Flexible (supported by Vector)
  - ECUM Fixed (similar to AUTOSAR 3.x, no **dedicated module*** from Vector)
- Manages fundamental ECU phases like
  - **OFF**, **UP** and **SLEEP**
  - The necessary transitions between them
- Initializes and de-initializes BSW, OS, and RTE
- Wakeup event handling
- Coordination of RUN requests from SWCs and BSWM
- Two startup and two shutdown phases

*) ECUM Fixed is **natively** supported by MICROSAR, but it is also possible to configure the **ECUM Flexible** automatically such that the behavior matches to the **ECUM Fixed** mode machine.

In order to do this, the BSWM has to be configured with the appropriate rules. Up until MICROSAR 4 Release 7, the configuration of the **ECUM Flexible** in the role of the **ECUM Fixed** is something the integration engineer has to configure manually. With MICROSAR 4 Release 8 and higher, the AUTO Configuration feature exists where the ESH (ECU State Handling) can be configured with a few clicks.

CW_26_2023 ; VI-SCER-Vector

Basic Software Mode Manager BSWM

BSWM

- The BSWM is a generic, **rule-based** service module
  - Used for mode arbitration and mode control
- It can have a configured set of **rules** containing **default states**, **conditions**, and **actions**
  - Each **condition** evaluates a set of **logical expressions***
    - Evaluation result for a rule is either TRUE or FALSE
  - **Action list(s)** must be given for at least one evaluation result per rule
    - For example: call existing APIs, do a notification, perform a user-defined callout
- Operation principle for rule evaluation
  - Caller's context (IMMEDIATE)
  - `BswM_MainFunction` context (DEFERRED)
- The `BswM_MainFunction` serves to implement
  - DEFERRED mode request ports, BSWM Timer, RTE Mode Switch Actions

*Logical expressions contained in conditions are composed of atomic expressions. This corresponds to the common way to handle conditions in boolean expressions like the "if" statement in the C programming language.

CW_26_2023, VES CEA Seitech

Basic Software Mode Manager BSWM

1. BSW Modes (Standardized C-API)

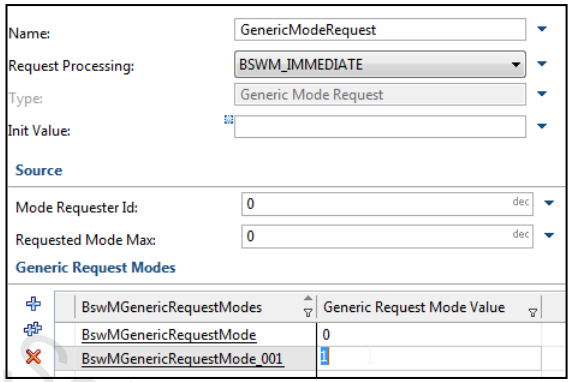
- `BswM_<BSW>_currentState/Mode`
  Standardized C language API
- Functionality built into the Basic Software. Can be *Mode Switch Notifications* from BSW Modules acting as Mode Managers, but also *Mode Requests*, e.g., by the LINTP to switch the LIN schedule in the LINSM

2. SWC Modes

- `Rte_Switch` / `Rte_Mode` API, AUTOSAR Interface, i.e., "Ports"
- The SWC notifies its mode switches to the BSWM over a *Mode Declaration Group,* either directly or indirectly. The BSWM rules which react on the Mode Switch can trigger an action list.

3. Generic Modes (not using the RTE)

- User defined C API, e.g.
  `BswM_RequestMode(MRqrId0,BswMGenericRequestMode_01)`

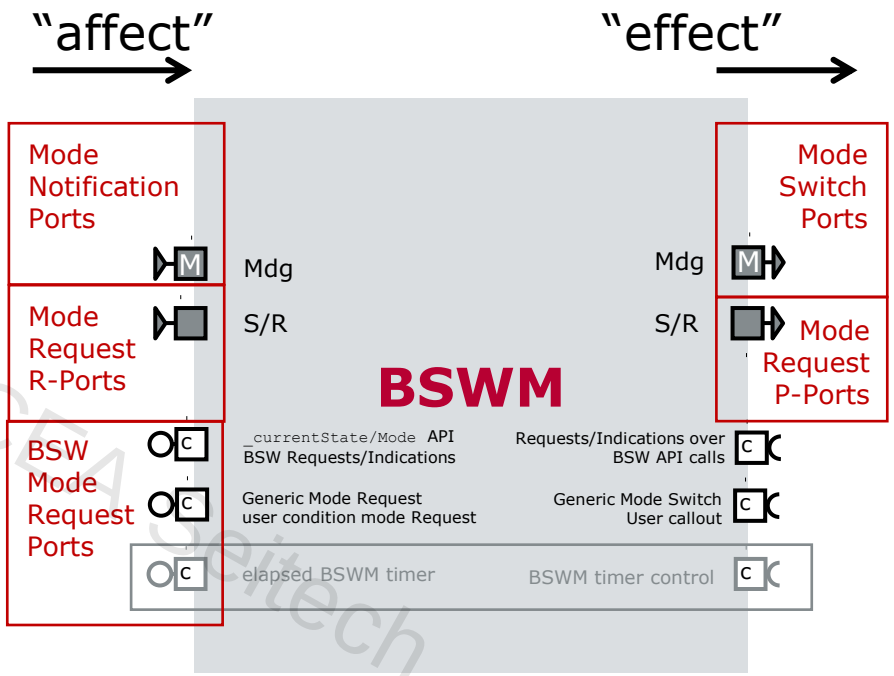


Generic Mode: If there is no built in BswM_<BSWM>`_currentMode/State` Notification like from the BSW and there is no Mode Switch Notification over the RTE, the Generic Mode can be used. Generic Modes are arbitrarily defined (not in terms of a Mode Declaration Group!).

CW_26_2023 : VI-S CEA Section

Basic Software Mode Manager BSWM

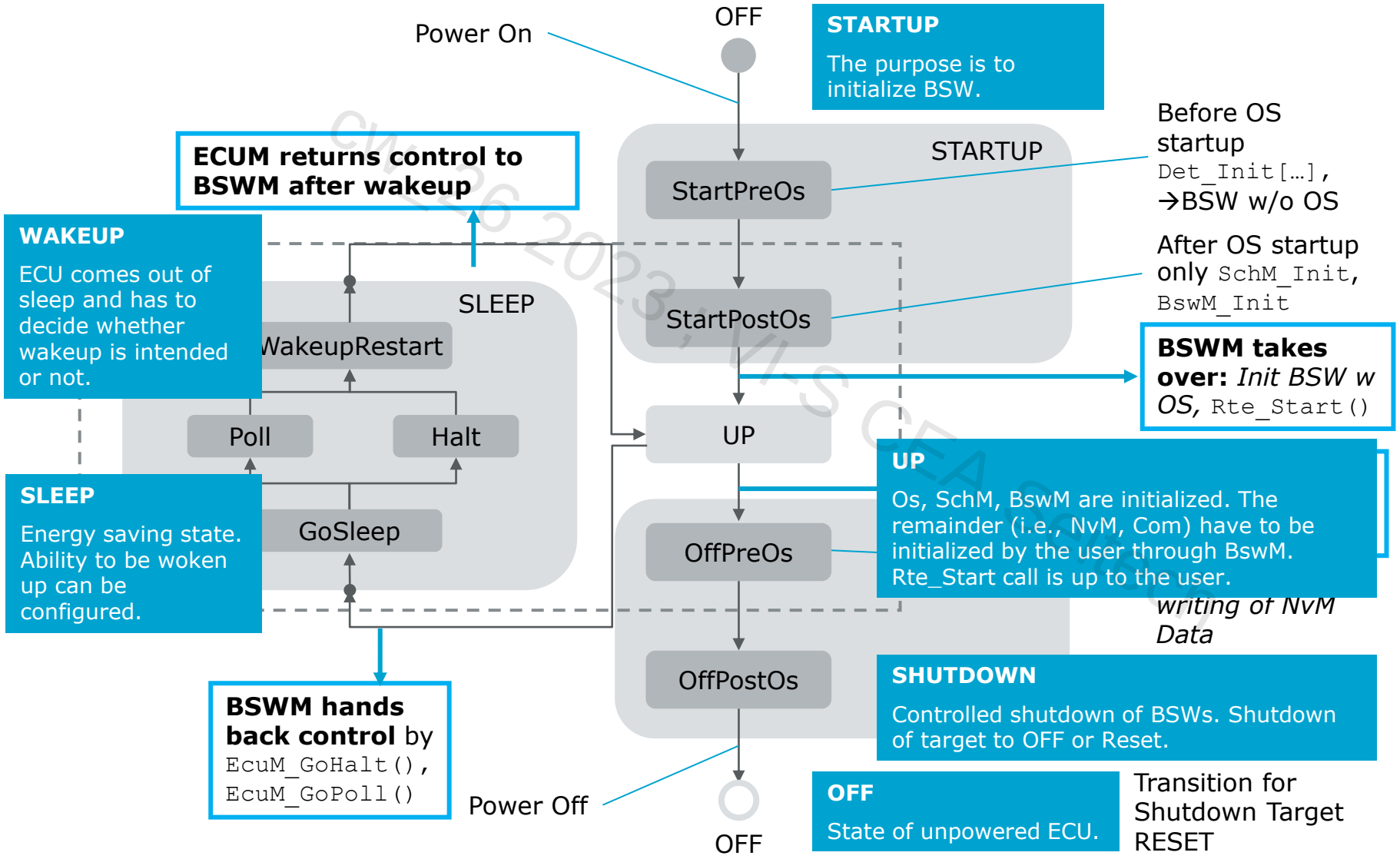
- **Mode Notification Port** (R-Port)
  - SWC indicates Mode to BSWM over a Mode Declaration Group
- **Mode Switch Port** (P-Port)
  - BSWM notifies Mode switch to a SWC using a Mode Declaration Group
- **Mode Request R-Port**
  - SWC requests Mode from BSWM over S/R
- **Mode Request P-Port**
  - BSWM requests Mode from a SWC Mode Manager over S/R



Module representation of the BSWM. The Ports labelled with "C" are only Standardized Interfaces or C APIs. Mode Ports (Mdg) or S/R Ports are AUTOSAR interfaces. Timers are only used internally in the BSWM; therefore they are printed in grey.

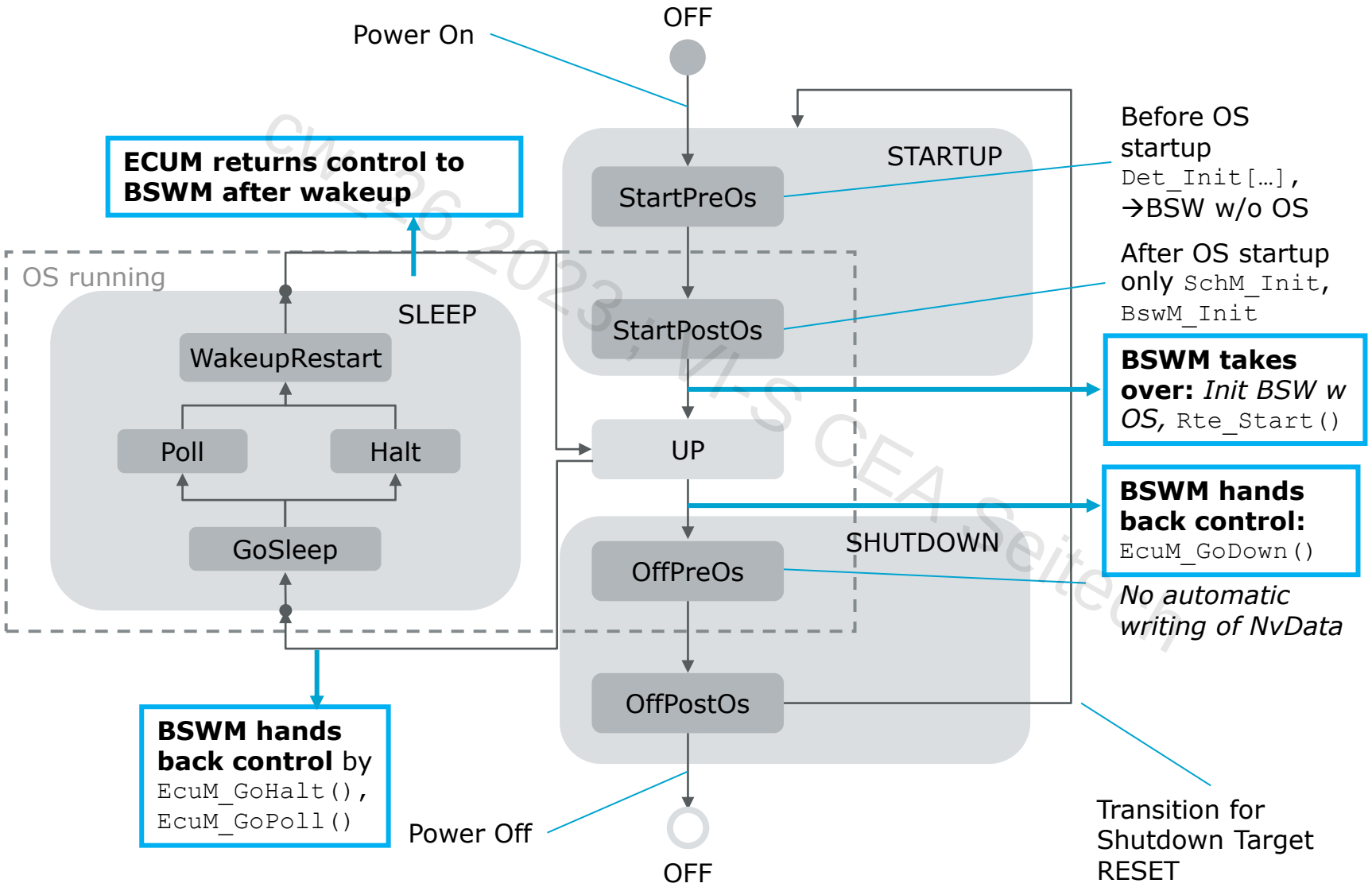
cw_26_2023 ; VI-S CEA Seitech

ECU State Manager (Flexible)



CW 26 2023, MS CEA 2023

ECU State Manager (Flexible)



CW 26 2023, MS CEA Seitzer

ECU State Manager (Flexible)

Here the ECU state machine is implemented as general modes and - during the UP Phase - under the control of the BSW Mode Manager module.

STARTUP PHASE: The purpose of this phase is to initialize BSW. It is divided into two parts, one before OS is started and the other after OS is started. In this phase, `SchM_Init()` and `BswM_Init()` must be called in the given sequence such that the StartPostOs already works based on the SchM Task(s) and an operating BSWM.

UP PHASE: This phase is entered after Os, SchM and BSWM have been initialized by the ECU State Manager. Please note that the restoring of NVRAM blocks or the communication stack initialization is not automatically done. Further, the RTE is not yet running as this is also something the user has to take care of. During UP phase, the ECU goes from State to State and from Mode to Mode, as modelled by the integrator-defined state machine and the help of BSWM. The BSWM implements the desired state machine which is required to restore and save NVRAM data, initialize and deinitialize the Communication stack and so on. Optionally, SLEEP modes can be entered from within the UP Phase of the ECUM. This phase must be requested by the application (explicitly or implicitly) whenever it is needed to keep the ECU awake.

SLEEP PHASE: This is an energy saving state. Typically no code is executed but power is still supplied and, if configured, the ECU is able to be woken up in this state. The sleep can either halt execution of the processor or poll for wakeup events. **WakeupRestart**: Wakeup is reached when the ECU comes out of **SLEEP** due to intended or unintended wakeup. (The **Wakeup validation protocol** allows to distinguish intended wakeup events from unintended wakeup events.)

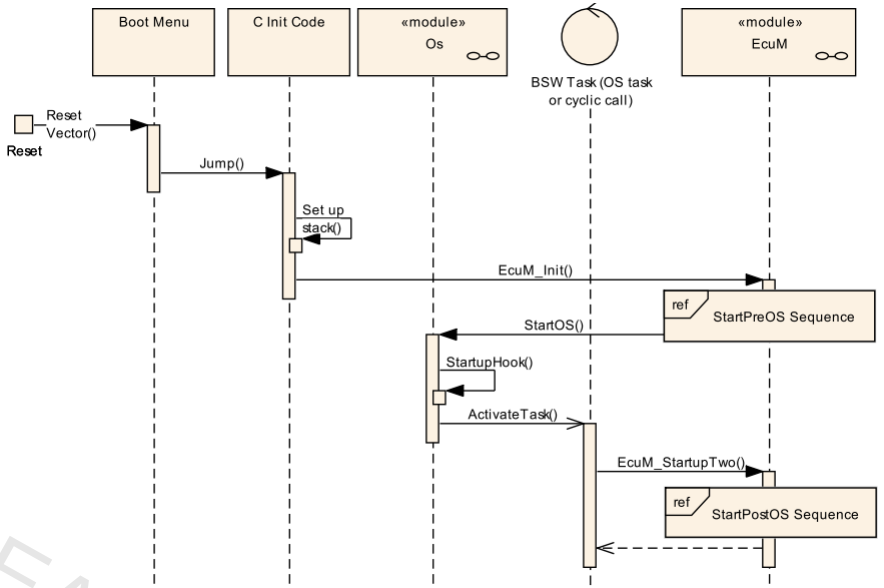
SHUTDOWN PHASE: This phase will be entered by the call `EcuM_GoDown()`. The OffPreOs state will then run (NVRAM blocks should be written before). At the end of this state, `ShutdownOS()` will be called. Then the sequence `ShutdownHook()`→`EcuM_Shutdown()` will start the OffPostOs state. The shutdown phase handles controlled shutdown of BSW and results in shutdown target for the ECU: Off or Reset

OFF PHASE: This is the state of an unpowered ECU. The ability to wake up from this state may be required but only for wakeup sources with integrated power control. In any case the ECU must be able to be started (like through a reset).

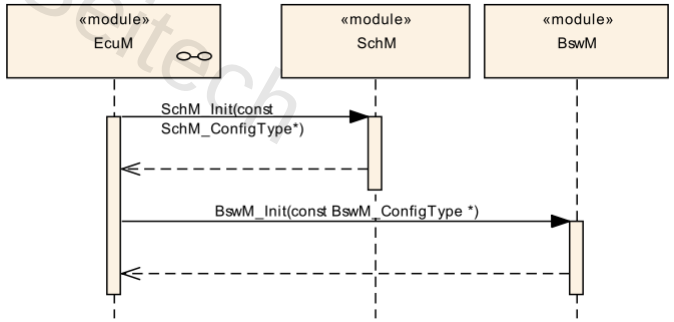
cw 36 2022 AUTOSEA oeitech

STARTUP PHASE

- **StartPreOs** Sequence begins after `EcuM_Init()` and ends with `StartOs()`
  - **StartPreOs** corresponds to STARTUP I in AUTOSAR 3.x
- **StartPostOs** Sequence runs in a task with the property `AUTOSTART=true`
  - it begins with the call of `EcuM_StartupTwo()`[1]
  - `BswM_Init()` executes the action list `BswM_ActionList_INIT_AL_Initialize`
  - **StartPostOs** corresponds [2] to STARTUP II in AUTOSAR 3.x



Overview of the startup phase



StartPostOs sequence: EcuM_StartupTwo()

source: SWS ECUM R4.1.1

cw 26 2023 ; VI-S CEA Seitech

STARTUP PHASE

1) The call of `EcuM_StartupTwo()` must be performed in an AUTOSTART Initialization Task which has the highest priority of all other (AUTOSTART) tasks in the system and is ideally non-preemptive. e.g.:

```
TASK(Init_Task)
{
  EcuM_StartupTwo();
  TerminateTask();
}
```

2) In contrast to **STARTUP II** in AUTOSAR 3.x, neither ECUM nor **StartPostOs** Sequence restore the NV Blocks or start the RTE automatically.

The restoring of NV Data is up to the user (literally the BSWM), as well as the call `Rte_Start()`.

Moreover, the permission for bus communication has to be granted by the user (or BSWM respectively):

The Call of `ComM_AllowCommunication()` is required in order to allow the `ComM_RequestComMode(ComM_User, COMM_FULL_COMMUNICATION)` to be successful.

The user has to configure the **BSWM** accordingly to manage these issues during the **UP PHASE**.

At the point where `BswM_Init` is called, the BSWM can already immediately evaluate a rule which reacts on the transition into the BSWM_INIT state and hence take over the ECU Management from this point in time.

Such a rule contains actions which can be the following:

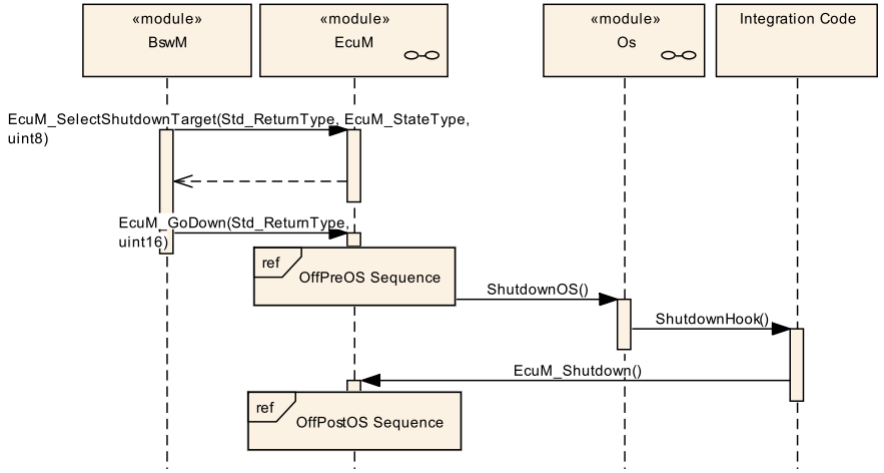
- Loading the NV Data over `NvM_ReadAll()`
- Granting the possibility to go into `COMM_FULL_COMMUNICATION`: `ComM_AllowCommunication()`
- Starting the RTE over `Rte_Start()` API

(In the simplest case this could be what is called `EcuM_InitListTwo()` in AUTOSAR 3.x, i.e. the InitListTwo would finally call `ComM_AllowCommunication` and start the RTE.)

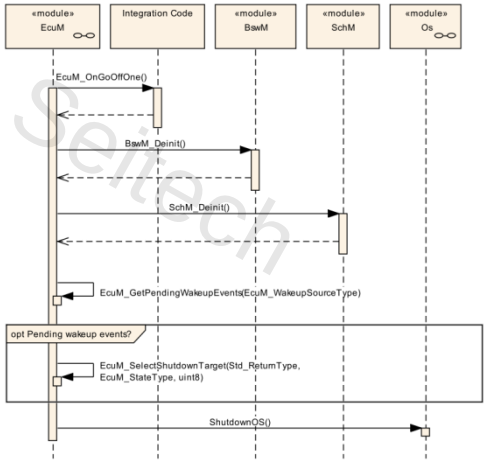
cw_26 2023 ; VLS CEA Gentech

SHUTDOWN PHASE

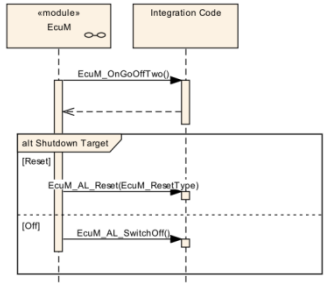
- **OffPreOs** Sequence begins by the BSWM calling `EcuM_GoDown(ModuleId caller)` and ends by the ECUM calling `ShutdownOs()`
  - **OffPreOs** corresponds to GO OFF I in AUTOSAR 3.x
- **OffPostOs** Sequence begins with the call of `EcuM_Shutdown()`
  - In the `ShutdownHook()` the integrator has to insert the call to `EcuM_Shutdown()`
  - It ends in `EcuM_Al_SwitchOff()`
  - **OffPostOs** corresponds* to GO OFF II in AUTOSAR 3.x



Overview of the Shutdown phase



OffPreOs Sequence



OffPostOs Sequence

source: SWS ECUM R4.1.1

cw_26 2023 ; VI-S CEA Seitech

SHUTDOWN PHASE

*) In contrast to **GO OFF II** in AUTOSAR 3.x, neither the ECUM itself nor the **OffPOstOs** Sequence stop the RTE or write NV Blocks.

The stopping of the RTE (`Rte_Stop()`) and the writing of the `NvM_WriteAll` Blocks is the job of the user.

The user has to configure the BSWM accordingly to manage these during the **UP PHASE** already.

If a handling for the RTE and NVM is needed later on, the last possibility is during `EcuM_OnGoOffOne()`, because directly afterwards the BSWM and SCHM become deinitialized.

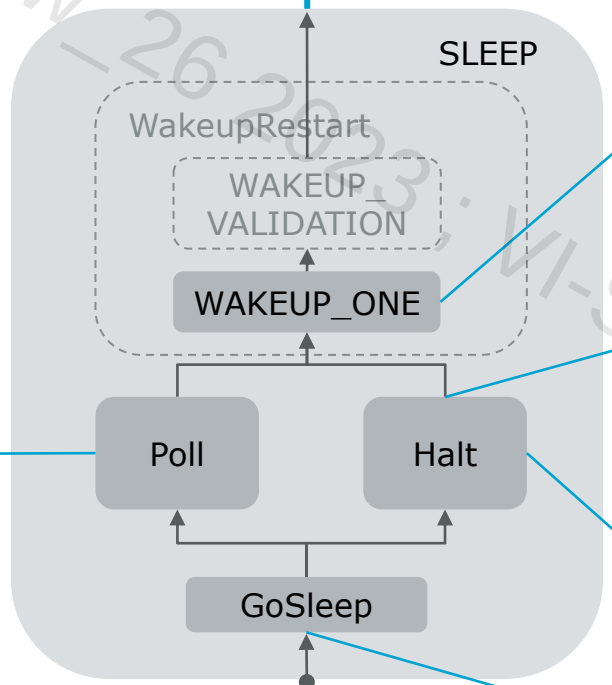
For the Shutdown, the **BSWM** could have a rule which reacts on an application state. If the application requests the shutdown e.g. over a Mode Request, the BSWM could

- Stop the RTE over `Rte_Stop()`
- Deinitialize the DEM (`Dem_Shutdown()`) and trigger the NVRAM to be written (`NvM_WriteAll()`)
- Call `EcuM_GoDown()` to shutdown the system

cw-20-2023 ; VI-s CEA Seitech

SLEEP PHASE: Poll or Halt

BSWM triggers `EcuM_GoHalt()`,`EcuM_GoPoll()` to continue Wakeup Validation , enters normal operation or `EcuM_GoDown()` to shutdown.



`EcuM_GlobalSuspend()`
`Mcu_SetMode(ECUM_SLEEP…`
`EcuM_GlobalRestore()`

EcuM_Mainfunction executes

- `EcuM_SleepActivity()`

Check for all Wakeup sources which support polling

- `EcuM_CheckWakeup()`

From normal operation or wakeup validation, BSWM triggers `EcuM_GoHalt()`, `EcuM_GoPoll()`

`EcuM_GlobalSuspend()`
`Mcu_SetMode(ECUM_NORMAL…`
`EcuM_GlobalRestore()`
`EcuM_DisableWakeupSources`
`EcuM_AL_DriverRestart()`,
`ReleaseResource(`
`ECUM_SLEEP_RESOURCE)`

`EcuM_CheckRamHash()`

- If failed: `EcuM_ErrorHook` `(no return)`

`EcuM_GlobalSuspend()`
`EcuM_GenerateRamHash()`
`Mcu_SetMode(ECUM_HALT…`
`EcuM_GlobalRestore()`

`EcuM_Disable/EnableWakeup`
`Sources(others/specific)`
`GetResource(ECUM_SLEEP_RE`
`SOURCE)`

cw_26_2023 : VI-SCEA Seitzh

SLEEP PHASE: Poll or Halt

GoSleep:

The ECUM prepares the hardware for going to sleep and setting the WakeUp sources, e.g., `EcuM_EnableWakeupSources`, as well as occupying the scheduler resource.

Poll:

The ECUM checks for pending *EcuMWakeupSourcePolling* wakeup sources cyclically by calling `EcuM_CheckWakeup()`. Auxiliary `EcuM_SleepActivity()` must be called for, e.g., updating the alarm clock. This activity will be performed through the EcuM_Mainfunction, which is still called while the scheduler resource of the OS is occupied.

Halt:

No more code is executed. Before halting the MCU, ECUM must invoke `EcuM_GenerateRamHash` and call `EcuM_CheckRamHash` before leaving Halt. On Multicore: only check the master core.

Wakeup Restart:

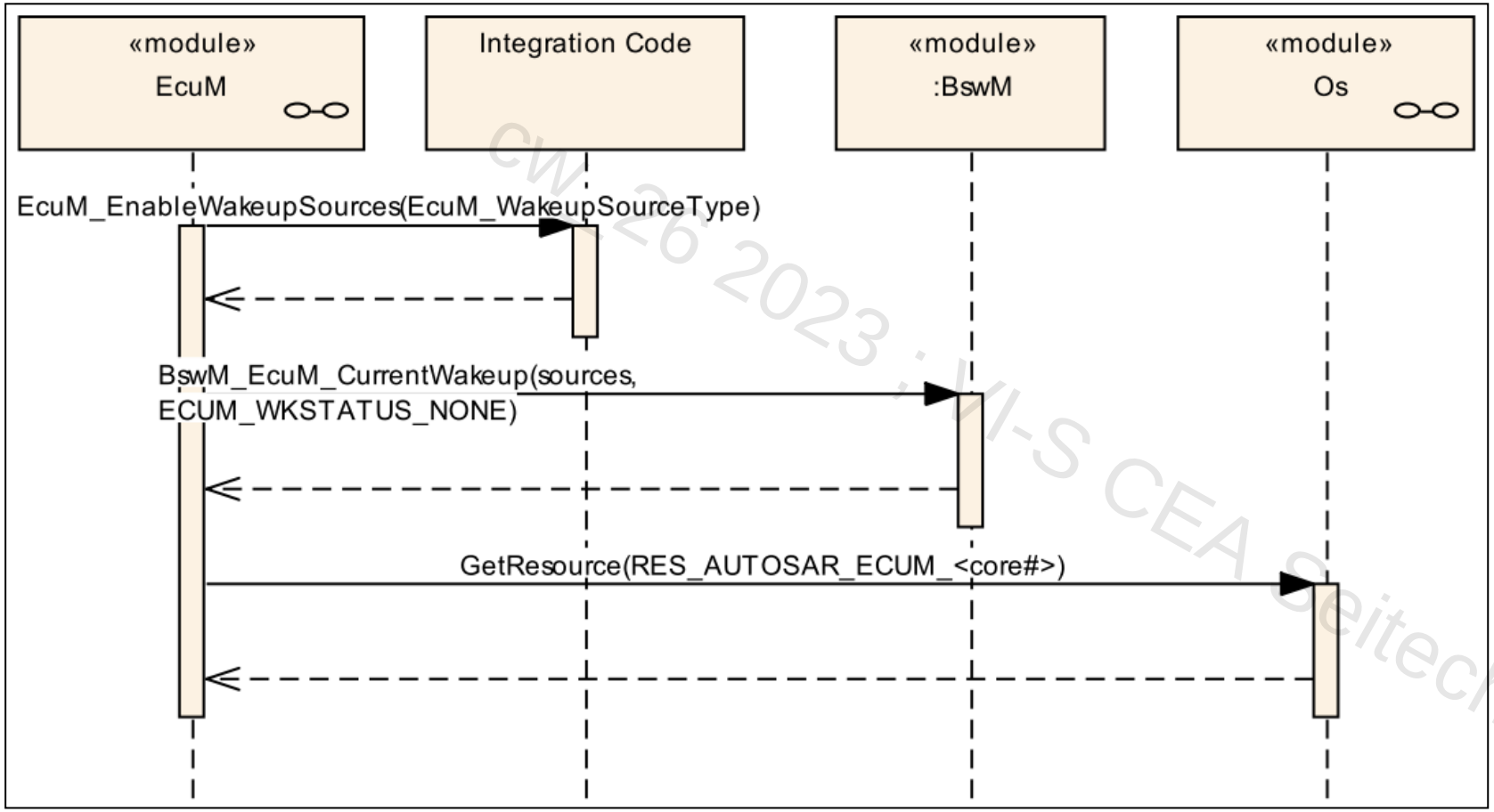
WAKEUP_ONE:

1) Restore MCU normal mode
2) Get Pending Wakeup Sources
3) `EcuM_DisableWakeupSources`
4) `EcuM_AL_DriverRestart`
5) Release scheduler resource

If after WAKEUP_ONE ValidatedWakeups is equal to False, the BSWM can trigger the transition back into GoSleep over `EcuM_GoHalt/EcuM_GoPoll` again, or even call `EcuM_GoDown`. Every pending event is validated independently (if configured) and the ECUM publishes the result as a mode request to the BSWM, which in turn can trigger state changes in the ECUM

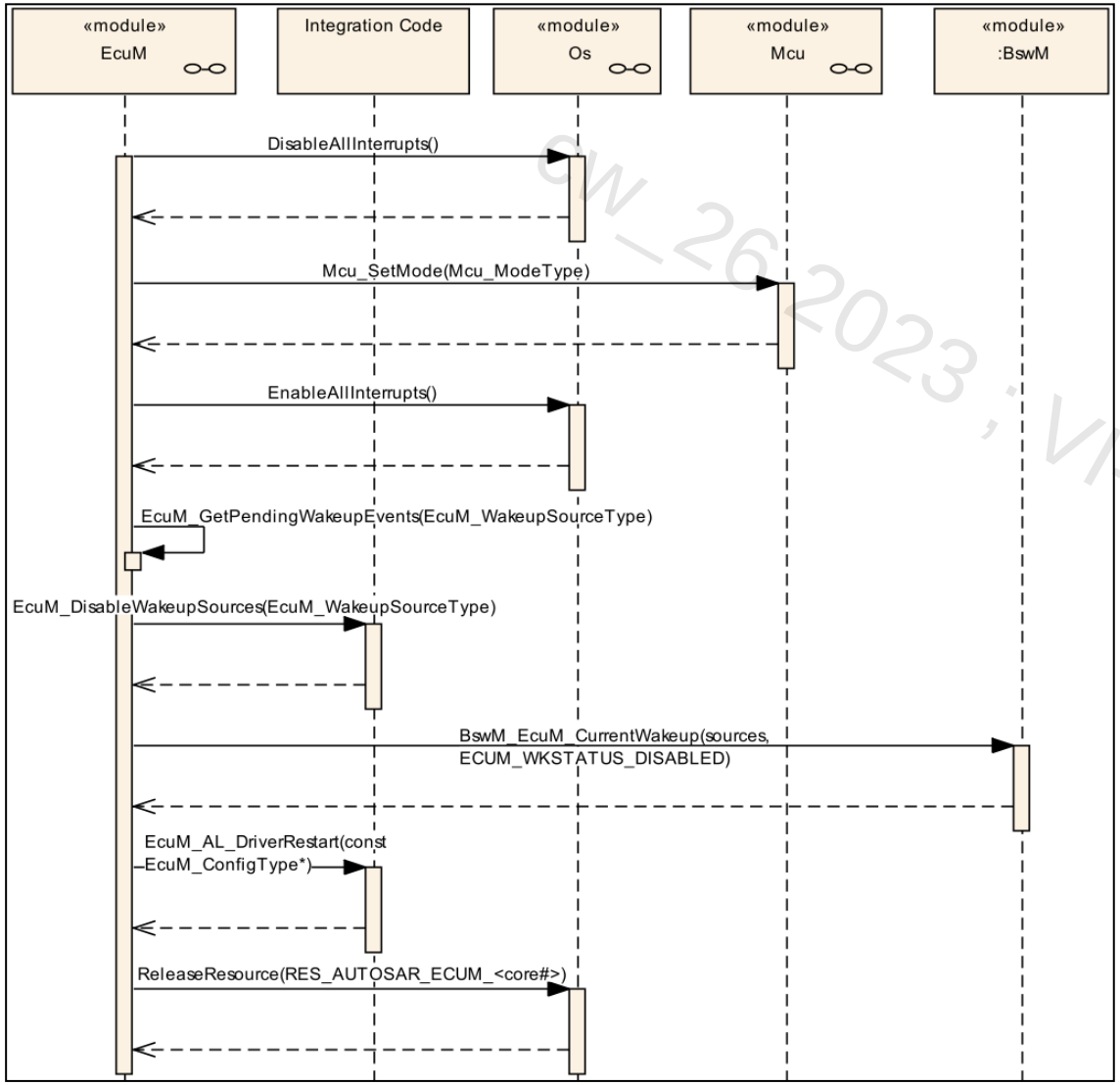
CW_26 2008, VI-S CEA Seitech

SLEEP PHASE: GoSleep



cw 26 2023 : W-S CEA Seitech

SLEEP PHASE: WakeupRestart



SW_26_2023 ; VI-S CEA Seitech

Agenda

- Mode Management
- **Wakeup Handling**
- BSWM configuration
- ECUM configuration
- Mode Management
- Network Management Algorithm
- Mode Manager Concept
- Service Mapping
- Coming up next

cw_26 2023 ; VI-S CEA Seitech

ECUM Wakeup Sources

- The AUTOSAR Wakeup procedure consists of two steps

  1. Check Wakeup Source & report Wakeup Event
  2. Wakeup Validation Protocol

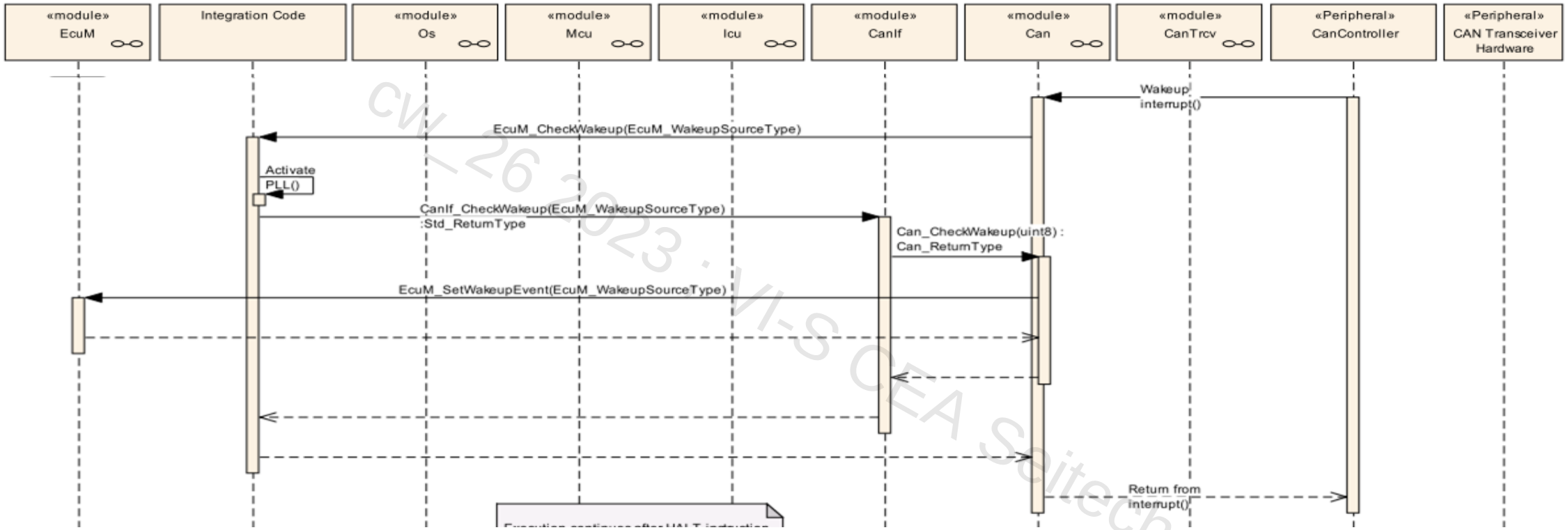
- Wakeup procedure independent from ECUM SLEEP PHASE
  - But used to wakeup from the SLEEP PHASE
- Each ECU owns a set of Wakeup Sources (maximum 32 sources)
  - Wakeup is coordinated by the ECUM
  - There are predefined Wakeup Sources
    - these Wakeup Sources do not need any validation

```
ECUM_WKSOURCE_POWER
ECUM_WKSOURCE_RESET
ECUM_WKSOURCE_INTERNAL_RESET
ECUM_WKSOURCE_INTERNAL_WDG
ECUM_WKSOURCE_EXTERNAL_WDG
```

  - Up to 27 user defined Wakeup Sources
- Each Wakeup Source either
  - Needs validation or
  - Is validated per default

FM_26_2/23, VI-S CEA Seitech

Cooperation of CAN, CANIF and ECUM



cw_26 2023 . VI-S CEA Saitech

- One possible wakeup scenario
  - CAN Controller Wakeup by Interrupt

Cooperation of CAN, CANIF and ECUM

Generic Elements: ECU firmware (file `EcuM_Callout_Stubs.c`)

- Functions that have to be implemented by the ECU supplier
- Function templates will be provided
- Intention of this firmware is to have the possibility to add project-specific code for mode changes
- In the exercise of this chapter the behavior shown above in function `EcuM_CheckWakeup()` has to be implemented

The ECUM handles the Wakeup over the APIs:

`EcuM_CheckWakeup(wakeupSource)` is called by the hardware from interrupt context. Depending on the origin of the interrupt, the call must be deferred to `<Bus>If_CheckWakeup(wakeupSource)` function.

The BusIf will then check the corresponding driver which is registered for the reported wakeup Source, e.g., `<Bus>_CheckWakeup` . The driver's CheckWakeup function will then report `EcuM_SetWakeupEvent()` in case it has detected a confirmed wakeup. This event will then be recorded in the pending wakeup events list.

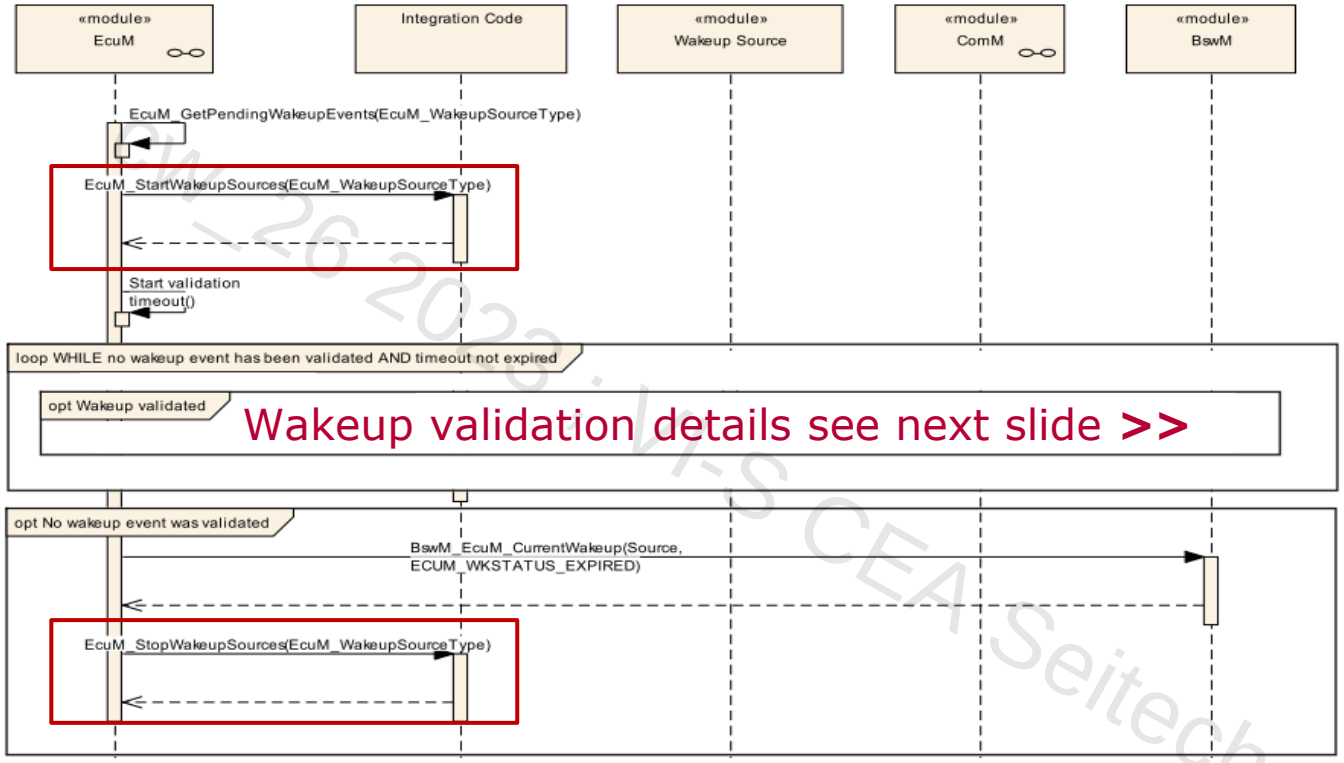
How to exclude Electromagnetic Interference (EMI):

The sequence from above may look a little awkward, but it is really required in some case (however it could be skipped in our case). If you have an SBC (System Basis Chip) or transceiver chip, and an Interrupt arrives which was caused by an ICU edge detection on the Rx pin at the µC Rx line, we need to double-check by reading out the SBC/transceiver state to confirm it was a wakeup event caused by the CAN bus and not an EMI spike between transceiver and controller on the Rx pin. Therefore the calling sequence is: `EcuM_CheckWakeup`→`CanIf_CheckWakeup`→`Can_CheckWakeup` or `CanTrcv_CheckWakeup`, respectively. This step is a validation on physical layer.

→This step is NOT YET the wakeup validation, which is a plausibility check on data link layer!

cw_26_2023_VI-6 CEA Seitech

Wakeup Validation



Nov 26 2020 - VFS CEA Seitech

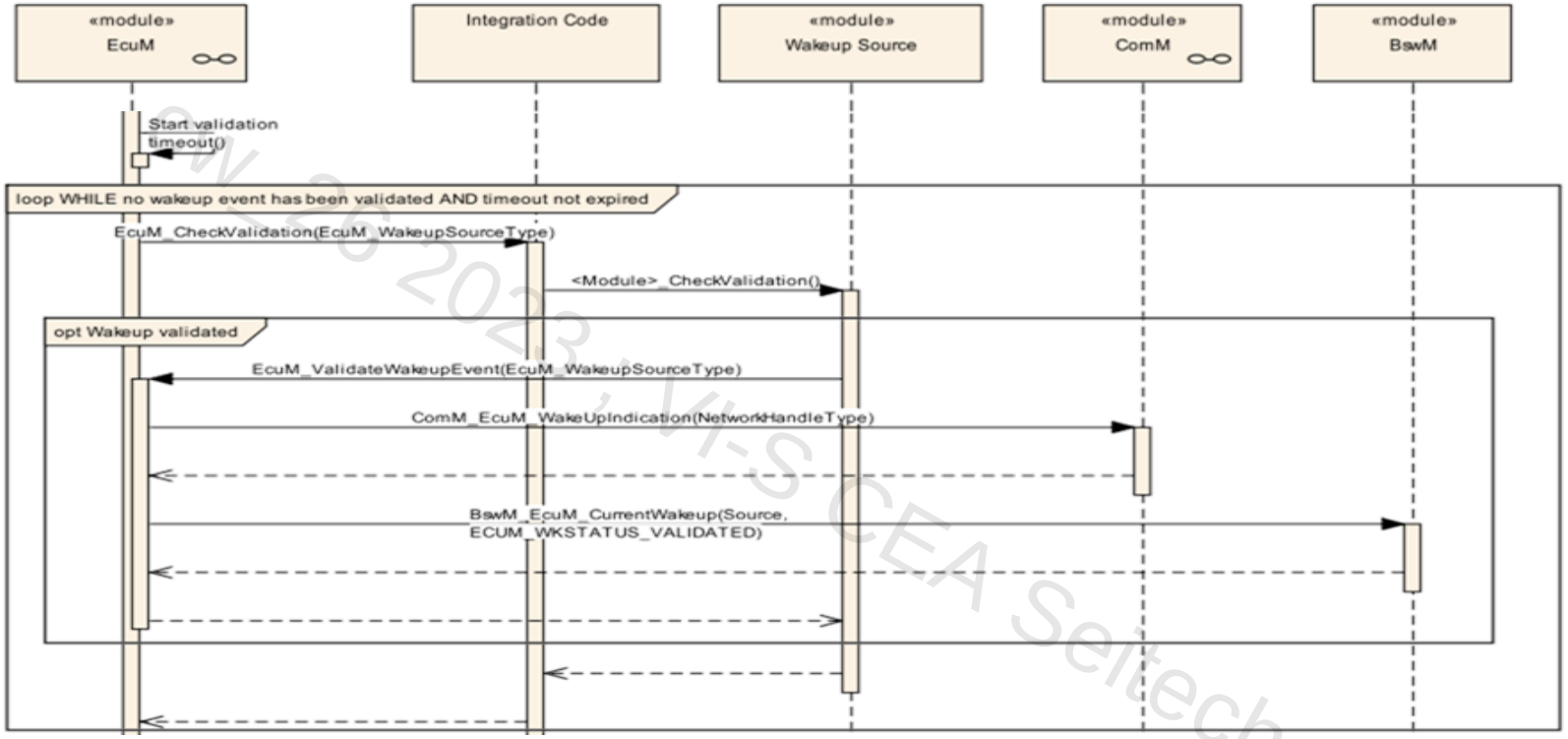
- The wakeup sources have to be started before the wakeup validation can proceed (`EcuM_StartWakeupSources`)
- after the failed validation the wakeup sources have to be stopped (`EcuM_StopWakeupSources`)

Wakeup Validation

- A wakeup source can have one of the following states:
  - `ECUM_WKSTATUS_NONE`: no wakeup was detected
  - `ECUM_WKSTATUS_PENDING`: wakeup was detected and reported but not validated
  - `ECUM_WKSTATUS_VALIDATED`: wakeup event was successfully validated
  - `ECUM_WKSTATUS_EXPIRED`: validation failed
  - `ECUM_WKSTATUS_ENABLED`: wakeup source is active and can report `EcuM_SetWakeupEvent()`
  - `ECUM_WKSTATUS_DISABLED`: wakeup source is disabled and cannot produce wakeup events.

cw_26_2023 ; VLS CEA Seitech

Wakeup Validation



CW 26 2023 - VH-S CEA Seltech

- CAN Wakeup Validation
  - can be enabled for each Wakeup Source separately (configure EcuMValidationTimeout !=0)

Wakeup Validation

As soon as the wakeup validation has succeeded from `EcuM_ValidateWakeupEvent()`, the event is removed from the pending wakeup events and added to the validated wakeup events.

If this cannot be accomplished, the Validation Timeout expires and the wakeup source would be considered expired (`ECUM_WKSTATUS_EXPIRED`).

Depending on the outcome of the wakeup validation, the BSWM will be informed about the result.

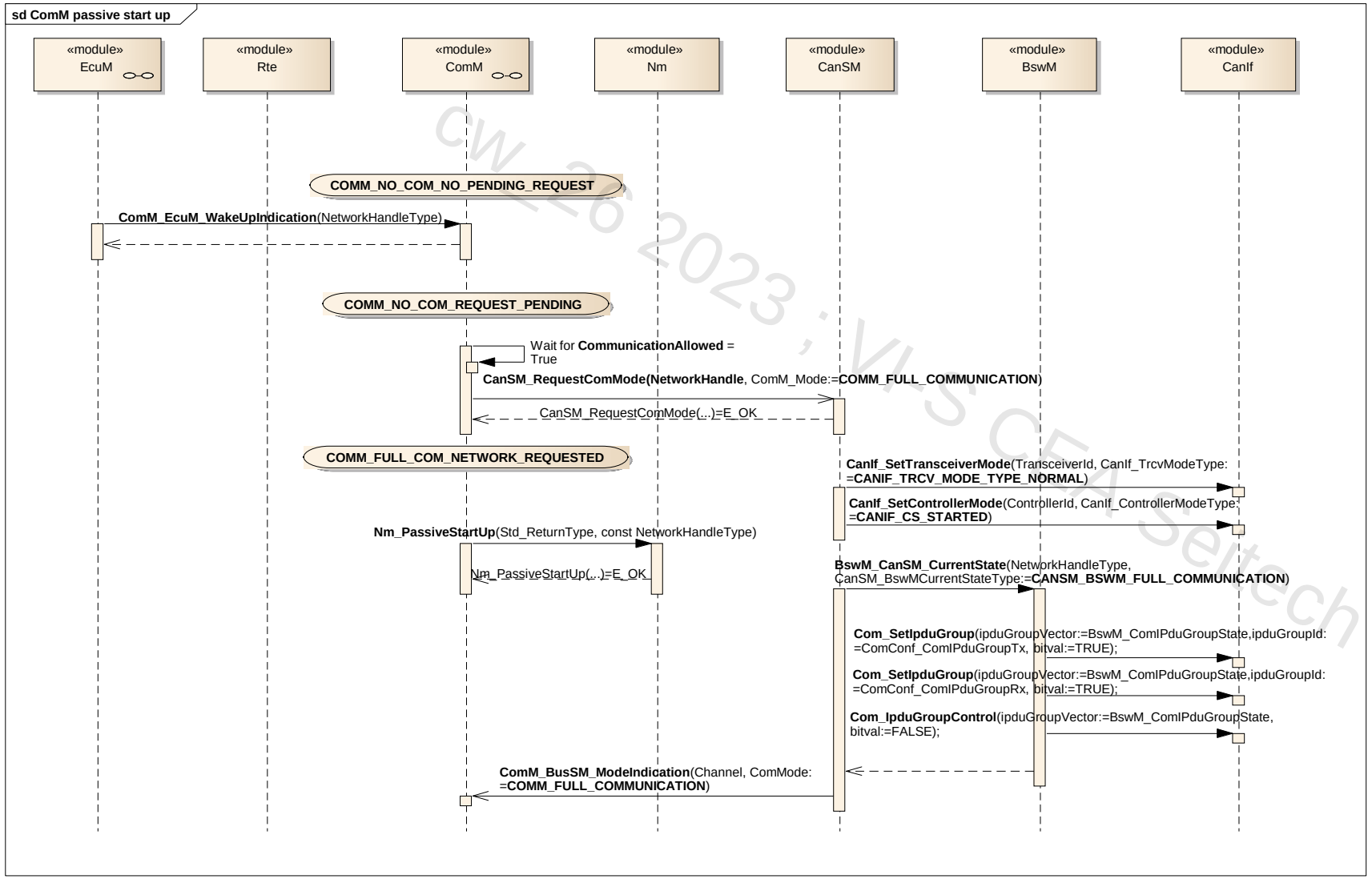
The Wakeup source is `ECUM_WKSTATUS_VALIDATED` in our example. If this is not successful, it will become `ECUM_WKSTATUS_EXPIRED`. The validated Wakeup events become notified to the BSWM over `BswM_EcuM_CurrentWakeupEvent(wakeupSource, ECUM_WKSTATUS_VALIDATED)`.

The call `BswM_EcuM_CurrentWakeupEvent(wakeupSource, ECUM_WKSTATUS_VALIDATED)` makes it possible to setup BSWM rules which perform actions on a wakeup source. This is especially the case when the wakeup source does not have a COMM Channel Reference and must be treated differently (because no `ComM_EcuM_WakeupIndication` can be used in this case)

If the wakeup source has a COMM Channel Reference, the ECUM further calls `ComM_EcuM_WakeUpIndication(NetworkHandleType Channel)` to notify the COMM to turn the communication channel on.

CW_26 2022 - V1 S CEA Setech

Cooperation of ECUM, COMM, BSWM, NM, CANSM at wakeup



cw 26 2023 ; VLS CEA Seltech

Agenda

- Mode Management
- Wakeup Handling
- **BSWM configuration**
- ECUM configuration
- Mode Management
- Network Management Algorithm
- Mode Manager Concept
- Service Mapping
- Coming up next

cw_26 2023 ; VI-S CEA Seitech

Auto Configuration

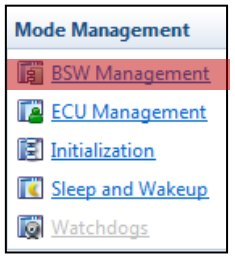
- Fundamentally different between AUTOSAR 3 and AUTOSAR 4
  - ECUM AUTOSAR 3 (ECUM Fixed AUTOSAR 3)
  - ECUM Flexible (AUTOSAR 4)

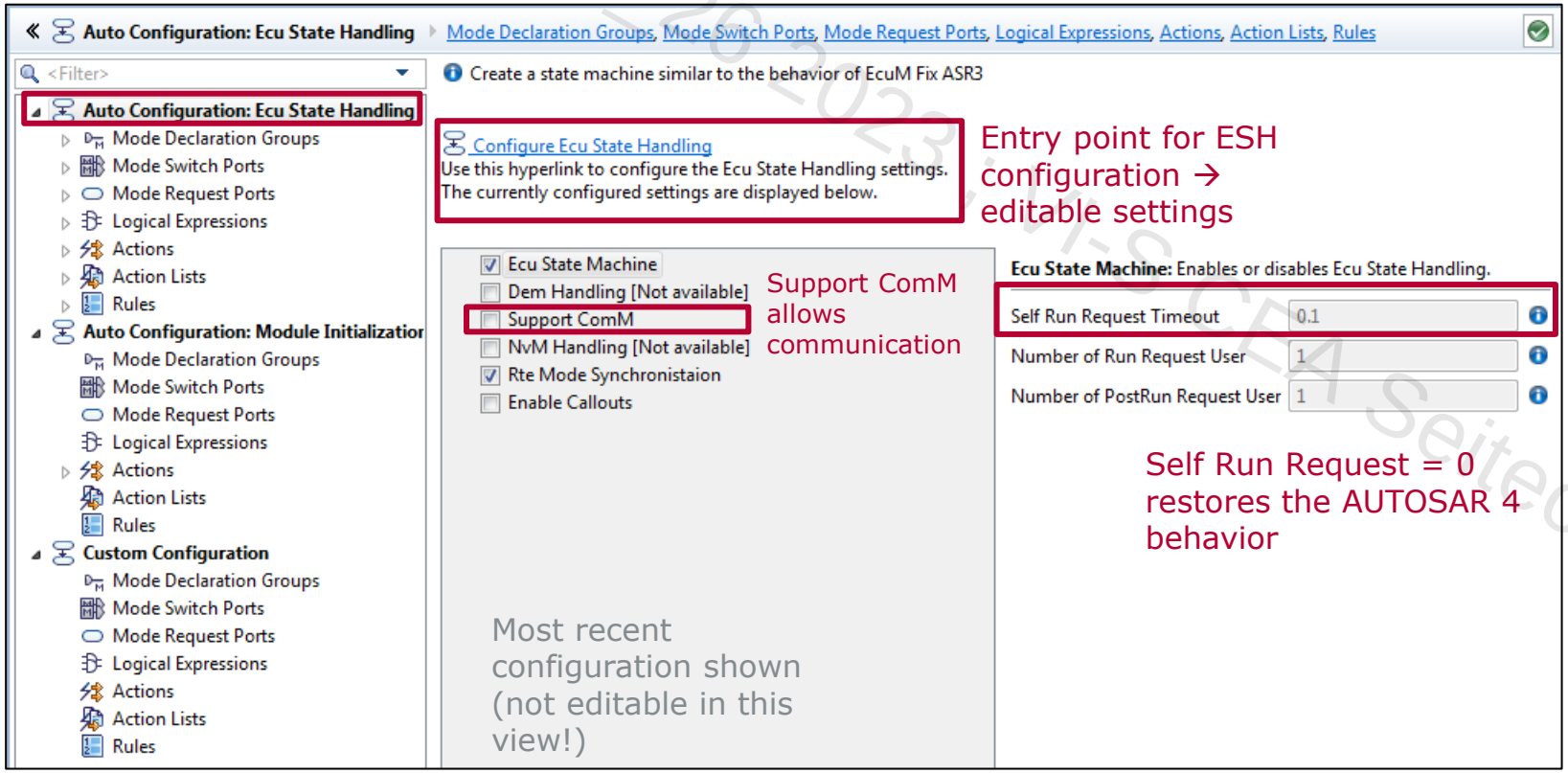
  However, for compatibility reasons, it would be nice to configure the AUTOSAR 4 ECUM + BSWM such that you obtain an ECUM Fixed according to AUTOSAR 3!
- This is what Vector's *Auto Configuration* feature of the BSWM can do!
- Auto configuration exists for
  1. ECU state handling
  2. Module initialization
  3. Communication handling

CW 26 2023, VI-S CEA Seitech

Auto Configuration: ECU State Handling (ESH)

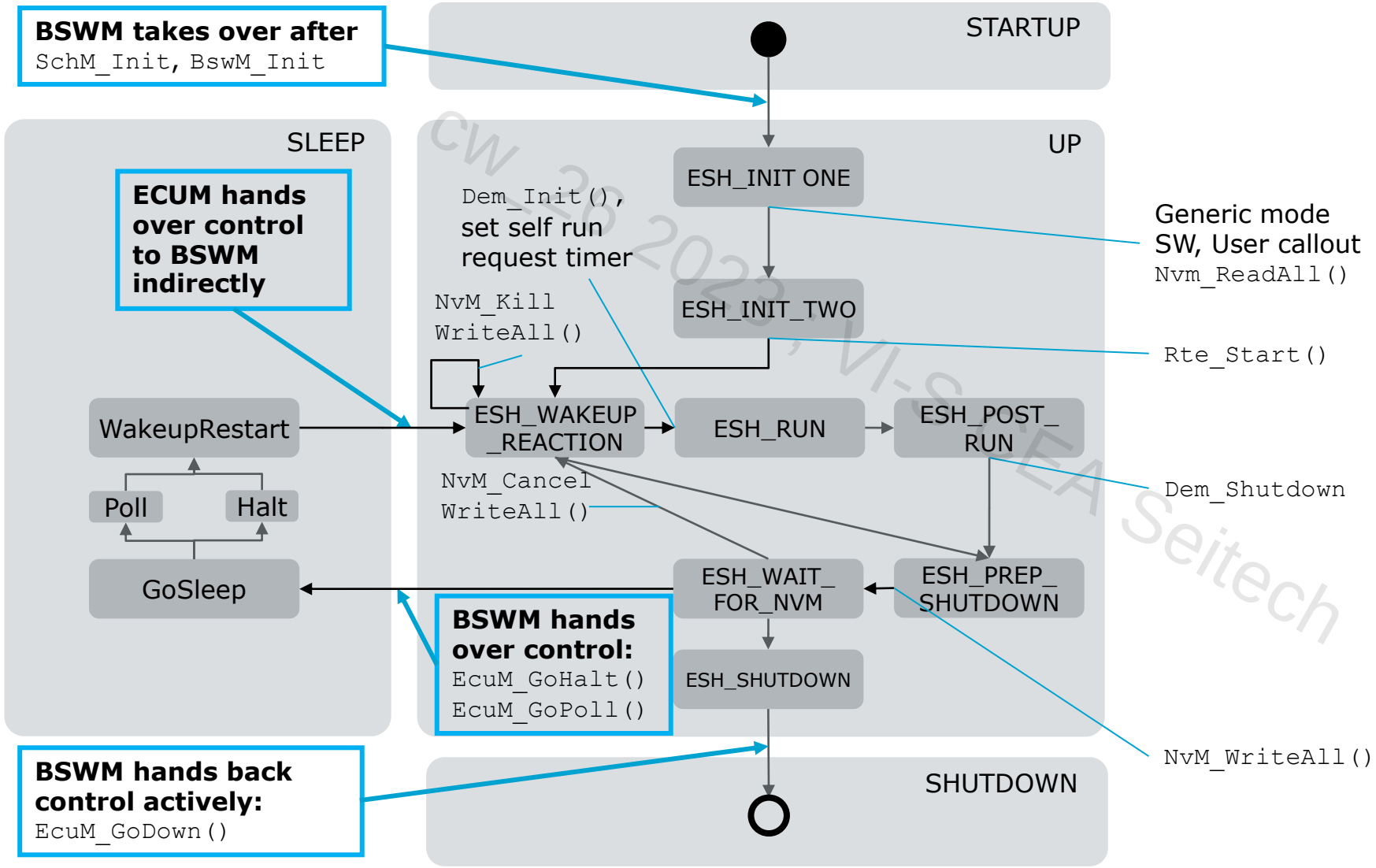
- The BSWM supports “pre-configured state machines”
- The Rules, Actions, Conditions, etc. they create are only a suggestion and may be edited by the integrator afterwards

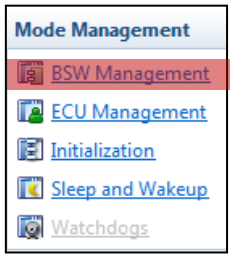




CW 26 2023 - VI-SCEA Seitech

Auto Configuration: ECU State Handling (ESH)



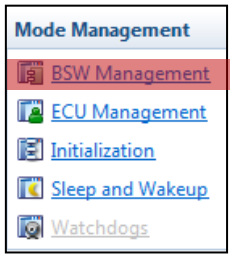


The ECUM Flex can be configured such that its behavior resembles the AUTOSAR 3 EcuM.

Wakeup Validation happens inside WakeupRestart.

CW 28 2022, VI-S-EA Seitech

Auto Configuration: Module Initialization

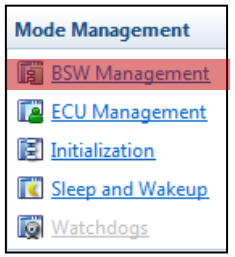


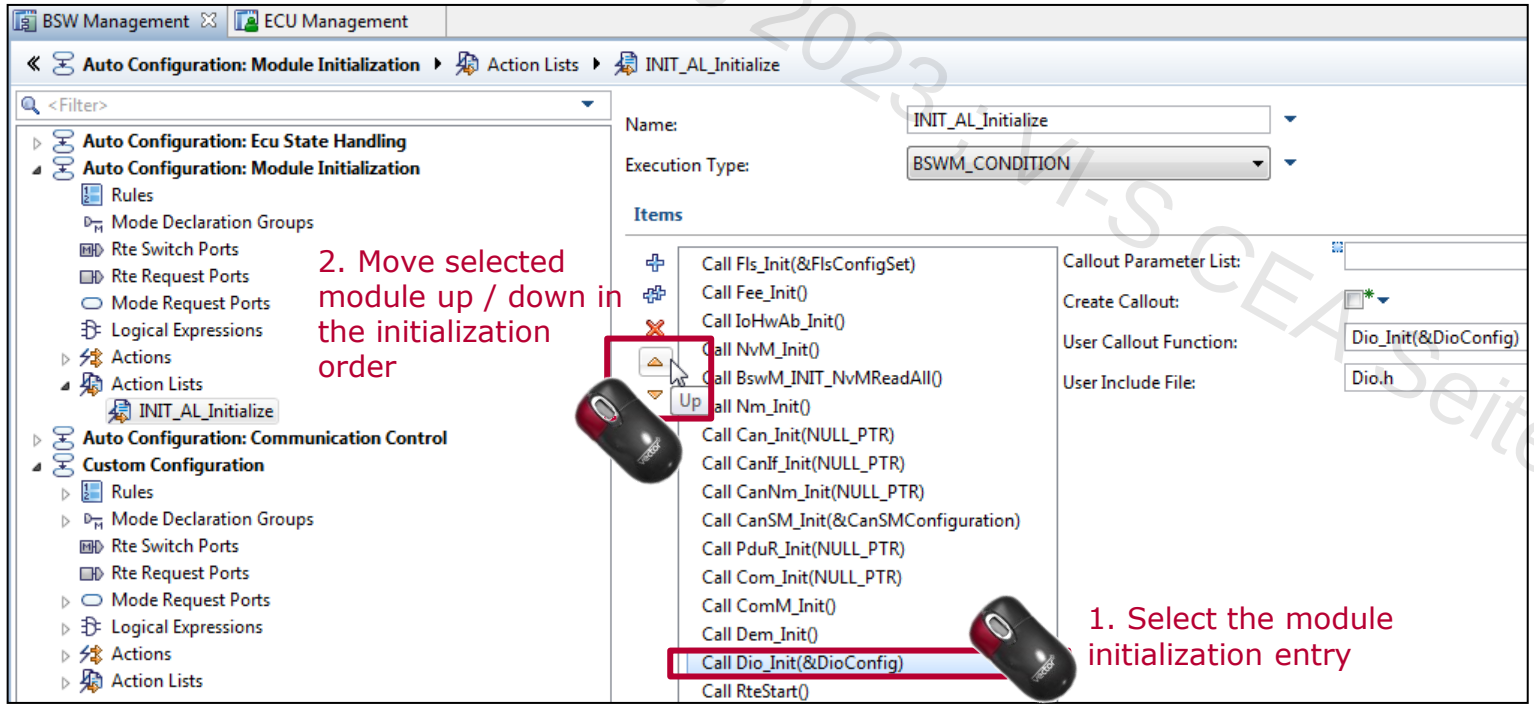
- Initialization of Basic Software Modules after OS Start
- BSWM lists all modules in the ECU configuration
  - Can be configured by setting the checkbox of each module separately
  - Modules listed alphabetically in the GUI, not chronologically. Actual initialization sequence can be seen in the **action list**
- BSW Modules which the BSWM does not know are listed in an extra category
  - `<BSW>_Init` Function and header must be entered by the integrator
  - Unknown modules will be initialized at the end of the **action list**
- The initialization order within the **action list** can be rearranged afterwards

CW_26_2023, VI-S CEA Seitech

Auto Configuration: Module Initialization

- After Auto Configuration, a reordering of BSW Module initialization steps is possible
- If the Auto Configuration for the Module Initialization is executed again, the default sequence will be restored





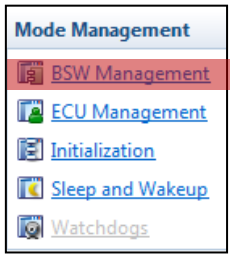
Rearrange the initialization order: Auto Configuration: Module Initialization → Action Lists → INIT_AL_Initialize, and move entries up/own using the icons in the Items dialogue.

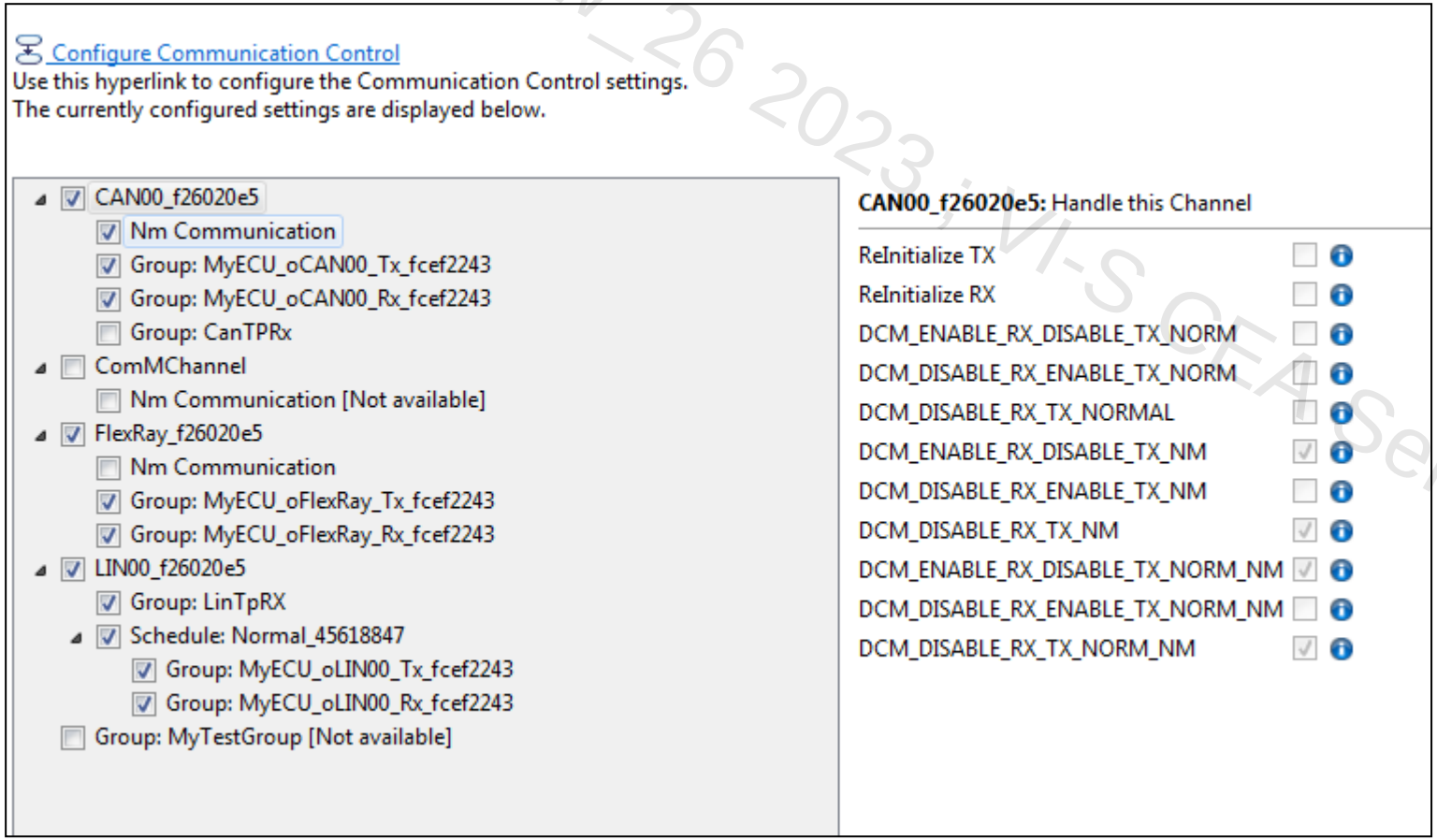
Auto Configuration: Module Initialization → Action Lists → INIT_AL_Initialize

ON 26 2023 M-S CERSeitech

Auto Configuration: Communication Assistant

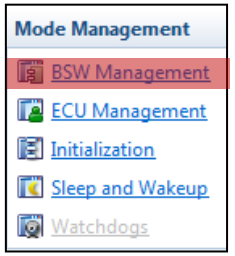
- BSWM now provides an Auto Configuration for Communication Handling.





cw_26 2023 / VI-S CEP Seitech

Auto Configuration: Communication Assistant

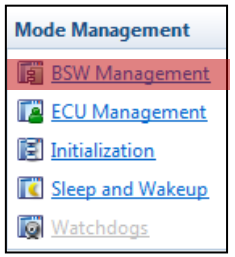


- Analyzing all PDU Groups and assigning them to a channel
  - PDU Groups which have PDUs of different channels can not be handled and will be listed as 'Not available'
- Switching of PDU Groups for CAN, LIN, FR, and J1939
  - User can choose to reinitialize I-PDU Groups or not (re-init TX / re-init Rx parameter)
  - CAN re-initialization will only performed if the Bus State changes from NO_COM to FULL _COM. In case of BUS_OFF or SILENT no re-initialization will be performed
  - CAN, FR: Enabling/Disabling the NM
  - DCM Modes can be considered for
    - Switching PDU Groups (CAN, FR)
    - Enabling/Disabling NM
  - DCM Modes are configurable per channel

CW_26 2023 - VFS CEA Seitech

Auto Configuration: Communication Assistant

- Partial Network Clusters (PNCs) are supported for CAN / FR
  - If a PDU Group can be assigned to a PNC, PDU Group is listed as a sub-feature of corresponding PNC and will be switched on/off depending on the PNC Status
    - *A PNC can only be determined if there are PNC mapping entries in the System-Description which are transferred to the FlatEcuc.arxml file*
- LIN
  - Only I-PDU Groups will be switched; no LIN Schedule handling is available
  - PDU Groups which contain only PDUs of the same Schedule will be switched depending on the schedule status



CW 28 2023 · VI-S CEA Seitech

i

▶Nm Mode will be forwarded to RM and DCM if the corresponding Node is configured there too

▶PDU Groups which contains only PDUs of the same Node will be switched depending on Node Status

▶PDU Groups which are determined as Broadcast Groups will be switched depending on Dcm Broadcast status

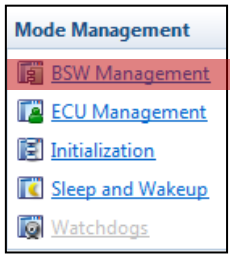
Agenda

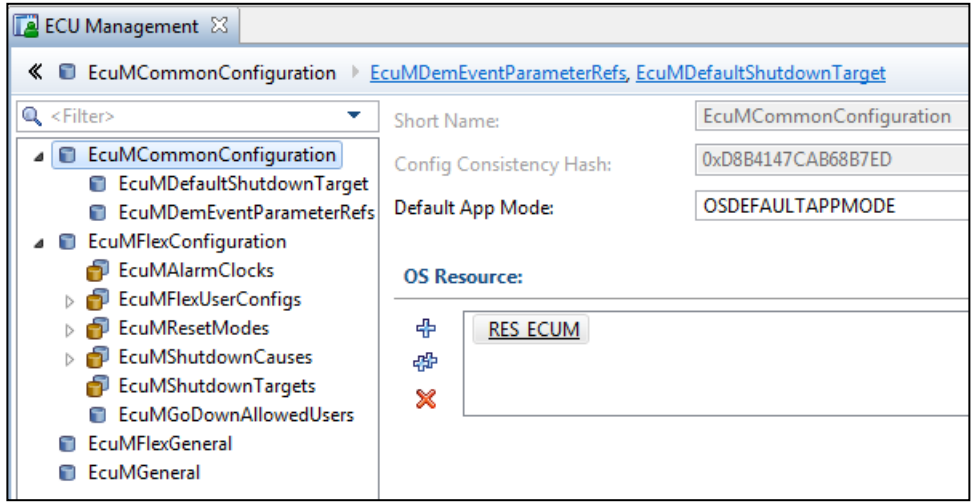
- Mode Management
- Wakeup Handling
- BSWM configuration
- **ECUM configuration**
- Mode Management
- Network Management Algorithm
- Mode Manager Concept
- Service Mapping
- Coming up next

cw_26 2023 ; VI-S CEA Seitech

ECUM General settings

- Since most of the work is done by the BSWM, the ECUM has become a very slim module
- However, a number of things can still be configured in the ECUM configuration
  - Os Application Mode
  - Scheduler Resource
  - DefaultShutdownTarget
  - Multicore operation
  - …





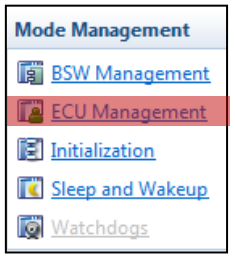
ECU Management: ECUM General settings

- Mandatory modules
  - AUTOSAR OS
  - MCU driver
  - DEM ("include DEM" switch has no meaning. Actual DEM usage will be determined by configured production errors)
  - COMM
  - SCHM

CW_26 2023 ; VI-S CEA Seitech

Shutdown related topics

- Reset modes specialize the Shutdown Target ECUM_STATE_SLEEP or ECUM_STATE_RESET
  - Identify the reason for an upcoming ECU reset
  - Selected by using `EcuM_SelectShutdowntarget`
  - Callout `EcuM_AL_Reset(EcuM_ResetType)`
  - User-specified implementation
- Select shutdown cause prior to shutting down
  - `EcuM_SelectShutdownCause`
  - ECUM_CAUSE_ECU_STATE - ECU state machine entered a state for shutdown
  - ECUM_CAUSE_WDGM - WdgM detected failure
  - ECUM_CAUSE_DCM - Dcm requests shutdown
  - User-configurable causes



E_DCM, ECUM_CAUSE_UNKNOWN, ECUM_CAUSE_WDGM, ECUM_CAU

EcuMShutdownCauses	Shutdown Cause Id
ECUM_CAUSE_DCM	3
ECUM_CAUSE_ECU_STATE	1
ECUM_CAUSE_UNKNOWN	0
ECUM_CAUSE_WDGM	2

G, ECUM_RESET_IO, ECUM_RESET_MCU, ECUM_RESET_WAKEUP

EcuMResetModes	Reset Mode Id
ECUM_RESET_IO	2
ECUM_RESET_MCU	0
ECUM_RESET_WAKEUP	3
ECUM_RESET_WDG	1

cw_26_2022, VI-S CEA Seitech

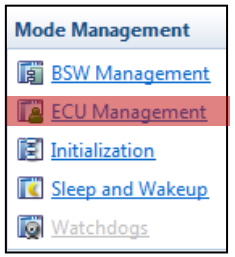
Shutdown related topics

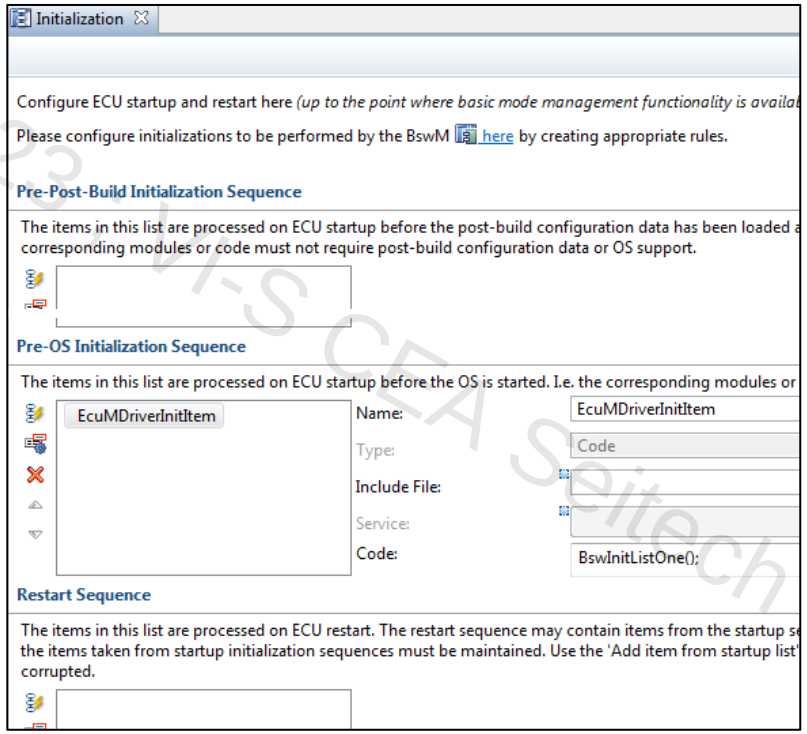
- The reset modes can be used to identify the reason for an upcoming ECU reset. A set of reset modes is defined by the AUTOSAR standard. Additional modes can be added by the configuration. The reset mode is passed over to the callout EcuM_AL_Reset(EcuM_ResetType) and the user can implement different ways to reset the ECU, depending on the reason for this reset.
- The Vector extension ECUM_RESET_WAKEUP is used as the reset mode in the case of a late wake-up event in the shutdown phase. If a wake-up occurs during the shutdown procedure, the shutdown target is changed by the EcuM to ECUM_STATE_RESET and the described mode is used.

Oct 26 2023 V1-S CEA Seitech

ECUM fundamental Initialization steps

- Startup Pre-OS
  - Pre-Post-Build and Pre-OS Initialization Sequences
- ECUM Restart Sequence
- Startup Post-OS
  - Handled by the BSWM

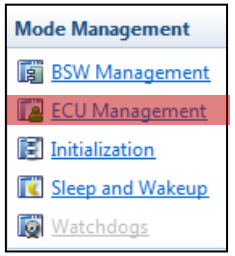




Initialization: The ECUM only initializes a subset which the BSWM cannot treat

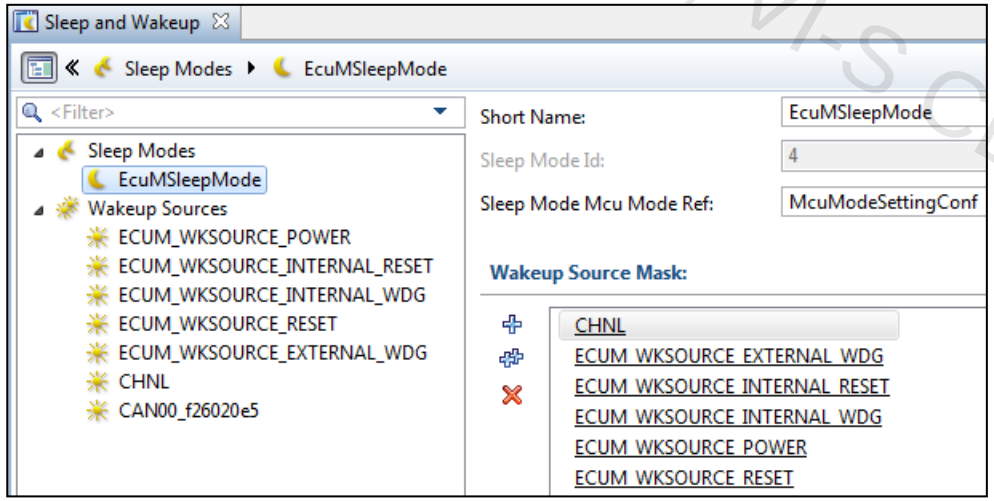
cw_26 2023 - W-S CEA Seitech

ECUM Sleep Modes and Wakeup Sources



- Wakeup Sources
  - These wakeup paths exist in the ECU
- Sleep Mode configuration
  - Each Sleep Mode has a MCU Mode and an associated set of possible wakeup sources to return into RUN

CW_26_2023 ; VI-S CEA Seitech



Sleep and Wakeup: Sleep Modes and Wakeup Source configuration

Agenda

- Mode Management
- Wakeup Handling
- BSWM configuration
- ECUM configuration
- ▶ **Mode Management**
- Network Management Algorithm
- Mode Manager Concept
- Service Mapping
- Coming up next

cw_26 2023 ; VI-S CEA Seitech

Communication Manager

COMM

- Controls the BSW regarding communication
- Coordinates bus communication requests from SWCs
- COMM cannot keep the ECU actively awake
  - ECUM Fixed or BSWM (ECUM Flexible) indicate to COMM when communication is allowed
    - `ComM_CommunicationAllowed` API
  - For the transition of the ECU into shutdown, ECUM (or BSWM respectively) must call
    - `EcuM_GoDown` API
- Uses NM of the networks to keep the networks awake
- Uses SM of the networks to control bus activity

cw 26 2023 ; VLS CEA Seitech

Communication Manager

Network Management

- Keeps the bus awake as long as an ECU requires communication
- Coordinates the transition to bus sleep

State Manager

- Enables and disables the bus communication (incl. transceiver handling)
- Controls COM I-PDU communication on each network indirectly over BSWM (BSWM operates the Com IPDUGroup API based on the CANSM currentState
- CAN Bus-off handling
- LIN Schedule Table handling

- If **ECUM Fixed** is used:

ECUM - Fixed needs to call and check `ComM_GetState()` before an ECU shutdown

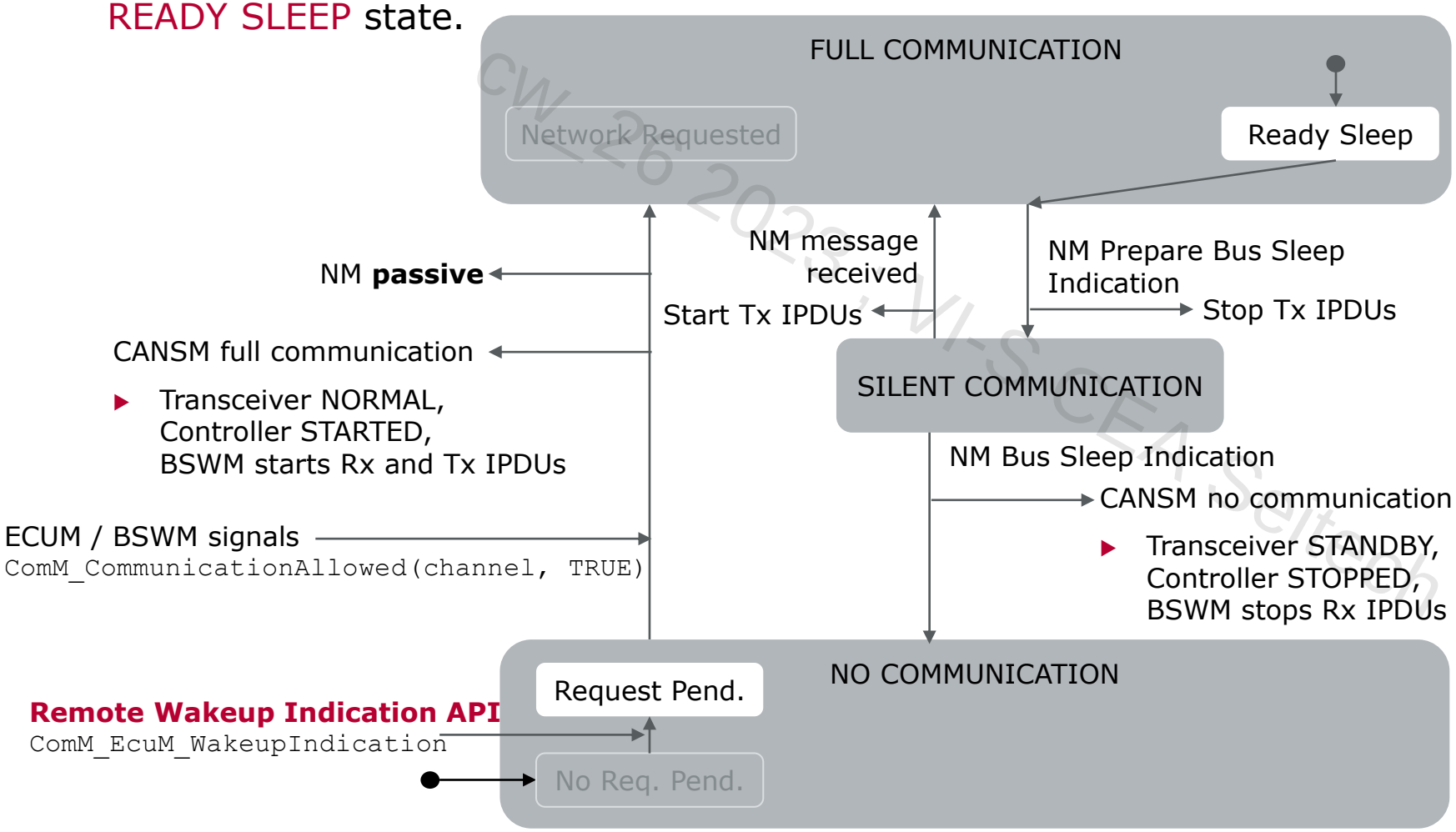
- If **ECUM Flexible** is used:

BswM should use `BswM_ComM_RequestMode()` indications to know when ECU can be shutdown. You can configure a rule which has the condition `ComMIndication == COMM_NO_COMMUNICATION` and has a corresponding action `EcuM_GoDown()`.

cw_26 2023 ; VH-S CEA Seitech

Communication Manager – External Wakeup

Always ready to sleep, cannot keep the network awake → start leads to READY SLEEP state.



CW_26_2026_VI-S_CEA_Scott_Jones

Communication Manager – External Wakeup

No communication (`COMM_NO_COMMUNICATION`)

ECU is not participating in bus communication. This mode is local for one channel, i.e., the ECU could communicate via other channels. Run mode must not be requested.

Silent communication (`COMM_SILENT_COMMUNICATION`)

Sending of PDUs for the corresponding channel is forbidden. Receiving PDUs is permitted.

Full communication (`COMM_FULL_COMMUNICATION`)

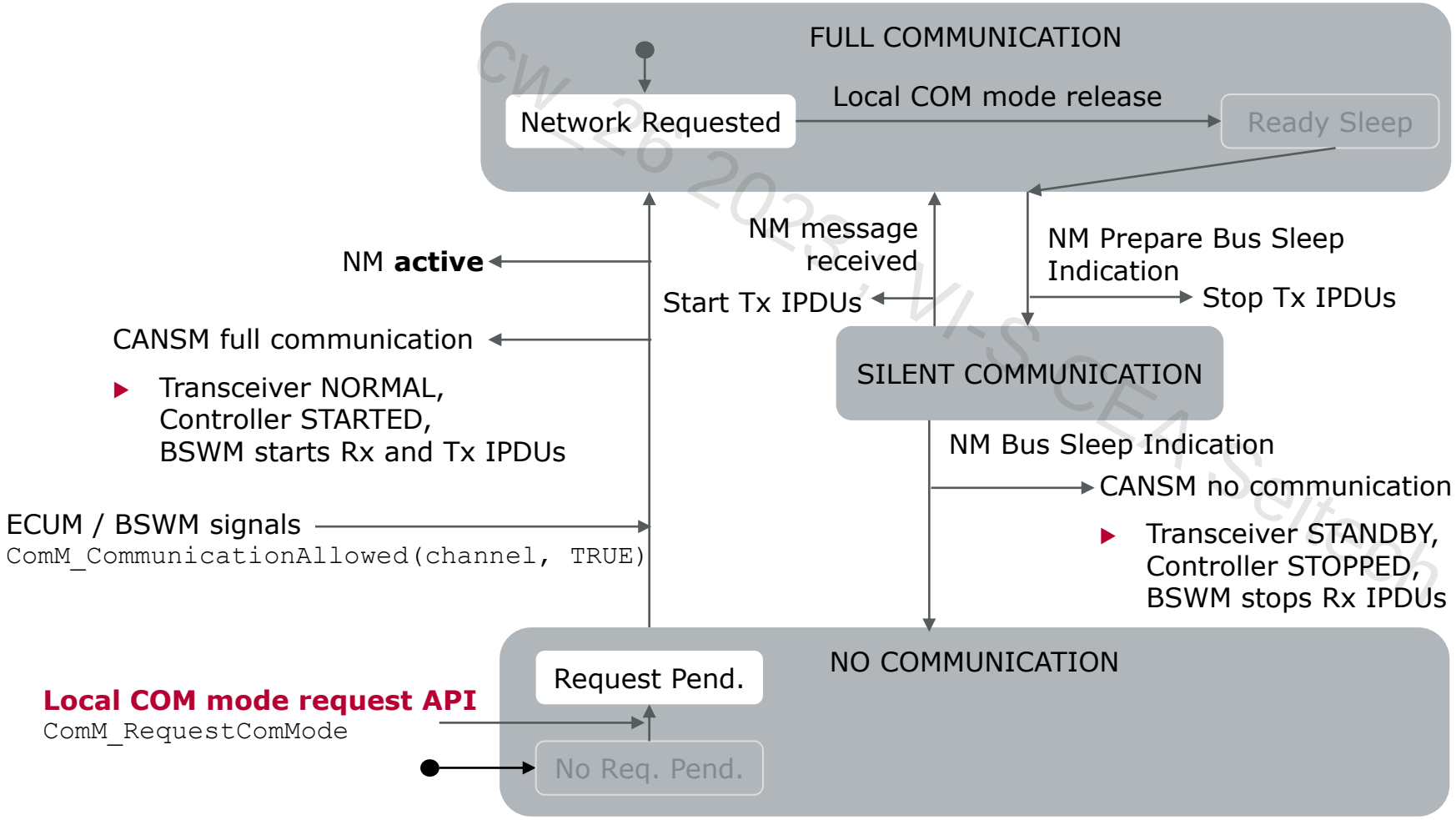
Transmission and Reception capability of corresponding channels is switched to on.

- **NM** passive indicates that the **NM** does not actively keep the bus awake. No **NM** messages are therefore transmitted (except when being in Repeat Message state for a short time).
- The **BSWM** controls the transmission and reception path over the state of the bus specific state manager module. To accomplish this, the **BSWM** uses the **COM** I-PDU-Group API.
- A transition to “internal communication request” – see following slide – is possible at any time if an internal communication request is made by a **SWC** or **DCM**

CW_26_2023 ; VI-S CEA Seitech

Communication Manager – Internal Wakeup

can keep the network awake → start leads to NETWORK REQUESTED state



A User **COM** Mode can be requested by a application **SWC**, by **DCM** during a diagnostic session or the **BSWM**

CW 26 2026 VI-S CEA Softies

State Manager

CANSM

- Translation of network communication mode requests
- Control of peripherals (indirectly over CANIF)
  - CAN Transceivers
  - CAN Controllers
- Control of IPDU groups of COM for
  - AUTOSAR version ≤ 3.1: CANSM directly controls IPDU Groups
  - AUTOSAR version ≥ 3.2: BSWM uses CANSM state to control IPDU Groups
- Handle the network mode via a separate state machine per network
- CAN Bus error management: Bus-off recovery via a separate state machine per network

CW_26 2023 . VT-S CER Seitecy

Network Management

•Keeps the bus awake as long as an ECU requires communication

•Coordinates the transition to bus sleep

State Manager

•Enables and disables the bus communication (incl. transceiver handling) in conjunction with the CANIF.

•Controls COM I-PDU communication on each network for AUTOSAR 3.1 and lower, but not for AUTOSAR 3.2 and higher.

•CAN Bus-off handling

•LIN Schedule Table handling

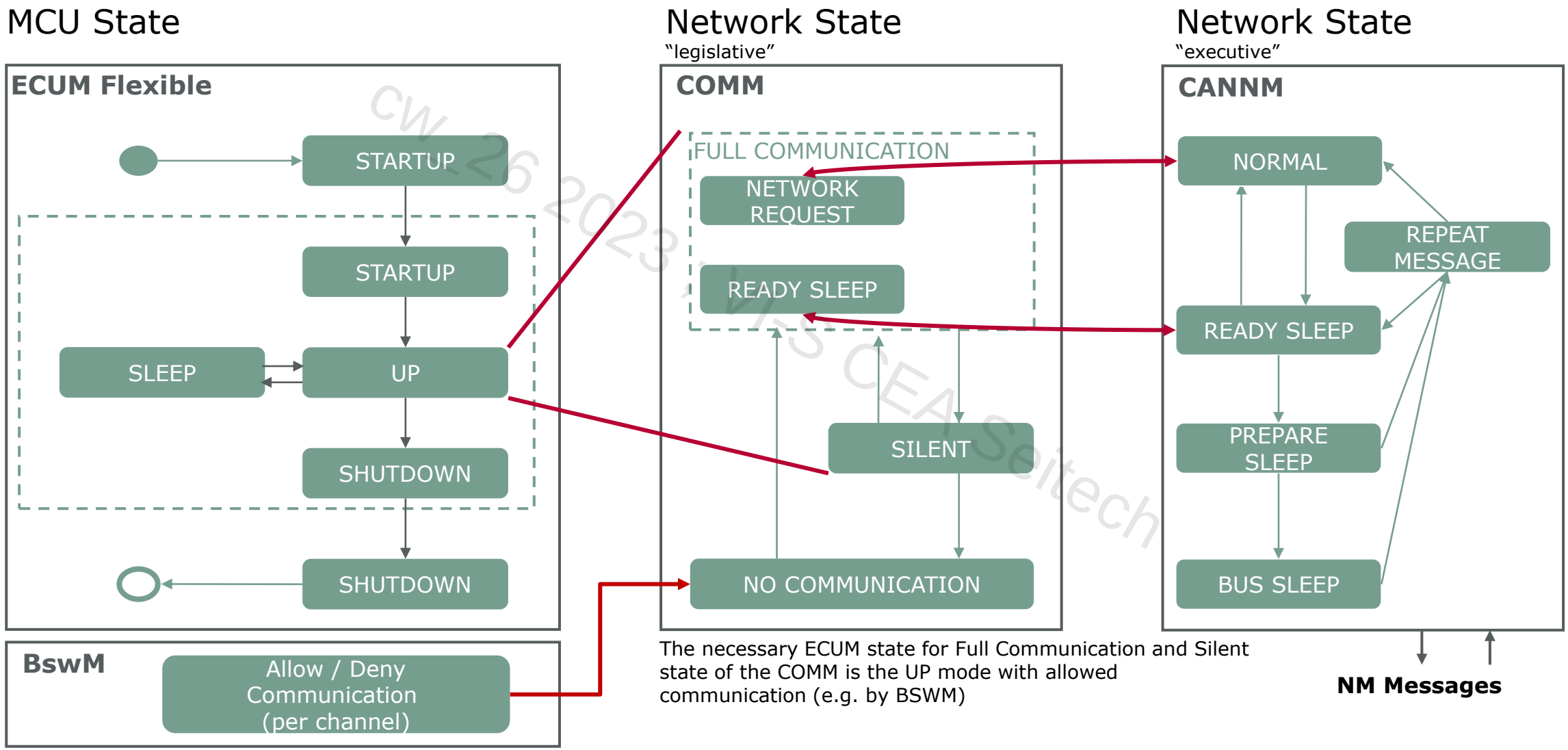
Network Management

NM

- Controlled transition of all ECUs into bus-sleep mode and vice versa
- Coordination functionality between different networks
- Node detection handling
- Abstraction of technology-specific NMs (CANNM, FRNM, LINNM, UDPNM) by a generic state machine

cw_26 2023 ; VI-S CEA Seitech

Interaction of ECUM, COMM, CANNM and BSWM



The necessary ECUM state for Full Communication and Silent state of the COMM is the UP mode with allowed communication (e.g. by BSWM)

cw 26 2023 VRS CEA Seitech

Network Management for CAN

CANNM

The AUTOSAR NM is a direct and distributed Network Management

- Direct NM - Each node has its own NM message
  - The NM message is reserved for the Network Management
  - The NM message signalizes that bus communication is needed
  - The NM message data is not relevant for the NM mechanism
- Distributed NM - All nodes are equal (there is no NM Master or NM Slave)
- Optional bus load reduction
  - Only two active nodes transmit NM messages
  - Avoids the bus load being dependent on the number of participating nodes
- Partial Networking
  - Impose logical network clusters (PNCs)

cw_26_2023, VUS CEA Seitech

Communication Manager – External Wakeup

The OSEK NM is also a direct and distributed Network Management.

The NM message data of OSEK NM is relevant for the NM mechanism as there are multiple NM message types

which can be differentiated by the message payload (Sleep Indication and Sleep Acknowledge fields)

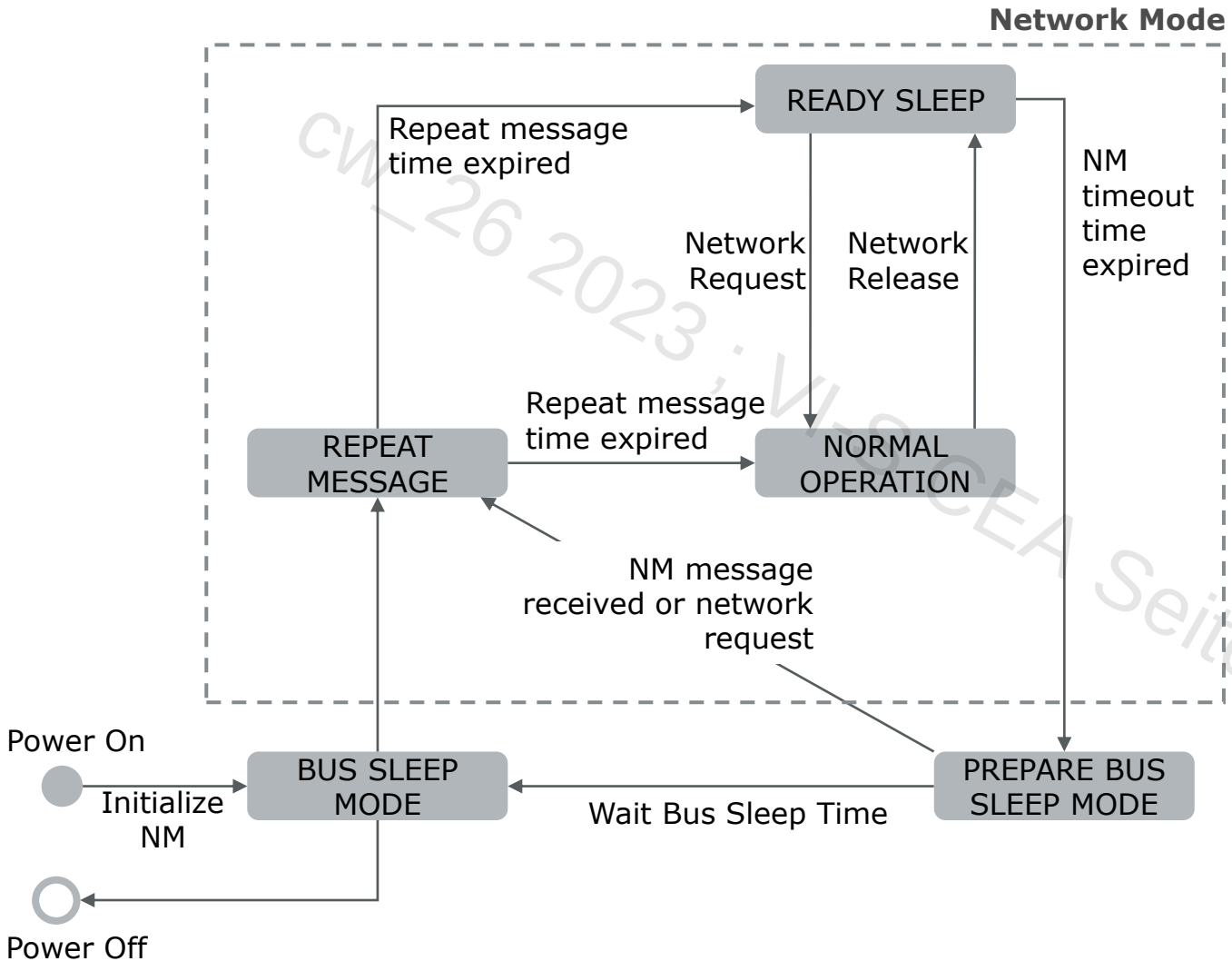
- Alive message
- Ring message
- Limphome message

With AUTOSAR NM, there is only one kind of NM message without payload, there is no Sleep Indication/Acknowledge field at all.

However, there is an NM API to allow access to a user or OEM-defined payload content which is not part of the NM algorithm.

cw_26_2023_VI-S CEA Seitech

Network Management for CAN



cw_26 2023 : VI-CEA Seite 9

During **Repeat Message** State, all modes that are not passive transmit their NM messages even if bus load reduction is active.

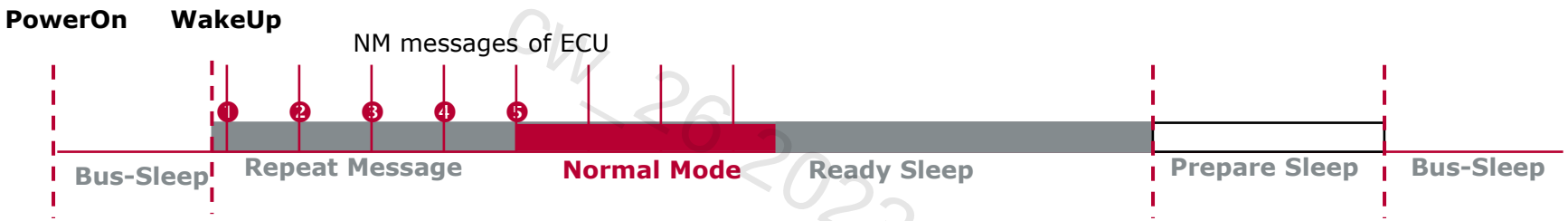
The Repeat Message State ensures that any transition from Bus Sleep or Prepare Bus Sleep to the Network Mode becomes visible to the other nodes on the network. The state can also be used to detect the present nodes (except passive nodes which do not transmit NM messages at all).

Agenda

- Mode Management
- Wakeup Handling
- BSWM configuration
- ECUM configuration
- Mode Management
- ▶ **Network Management Algorithm**
- Mode Manager Concept
- Service Mapping
- Coming up next

cw_26 2023 ; VI-S CEA Seitech

Basic Mechanism – Single ECU (CAN)



cw 26 2023 ; VI-S CEA Scitech

Repeat Message
Each ECU transmits its own NM message cyclically for NM_REPEAT_MSG_TIME.

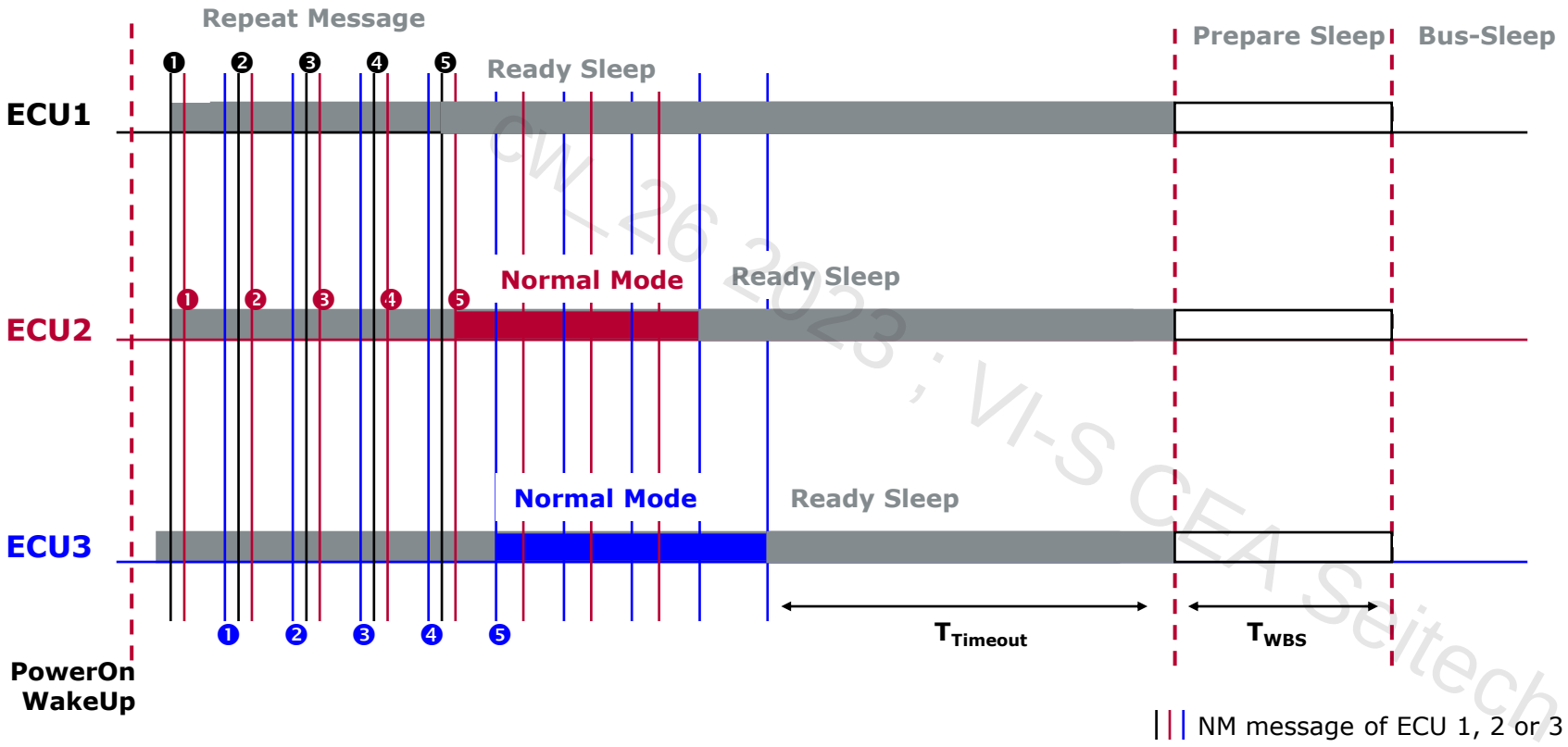
Ready Sleep
ECU is ready to sleep, no NM message transmission, restart of timeout timer upon NM message Rx.

Normal
NM message transmission and restart of timeout timer upon NM message Rx and Tx.

Prepare Sleep
If NM_TIMEOUT_TIME expired and no NM message has been transmitted or received meantime.

Bus-Sleep
After NM_WAIT_BUS_SLEEP_TIME transition to bus-sleep mode.

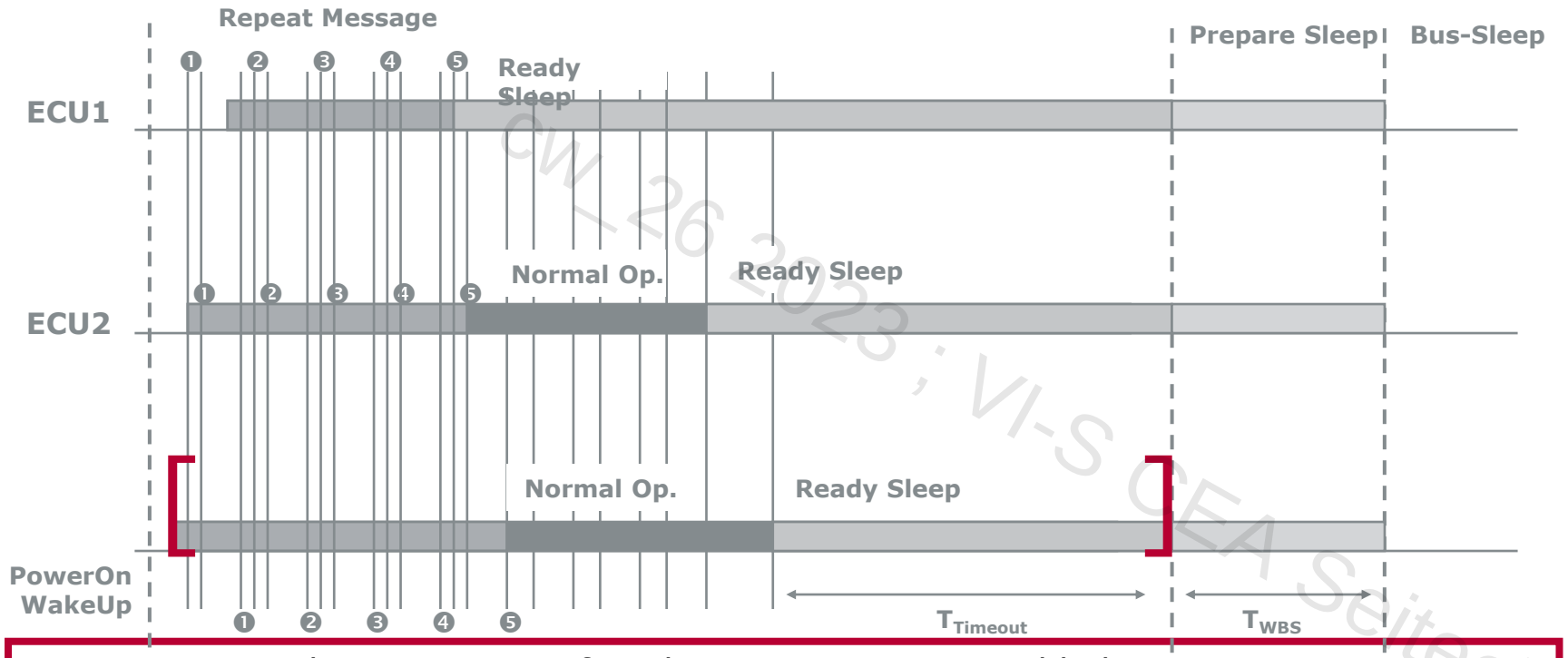
Basic Mechanism – Network (CAN)



CW 26 2023 ; VI-S CEA Seitech

Repeat Message: Each ECU transmits its own NM message cyclically for NM_REPEAT_MSG_TIME.
Ready Sleep: ECU is ready to sleep, no NM message transmission, restart of timeout timer upon NM message Rx.
Normal: NM message transmission and restart of timeout timer upon NM message Rx and Tx.
Prepare Sleep: If NM_TIMEOUT_TIME expired and no NM message has been transmitted or received in the meantime.
Bus-Sleep: After NM_WAIT_BUS_SLEEP_TIME transition to bus-sleep mode.

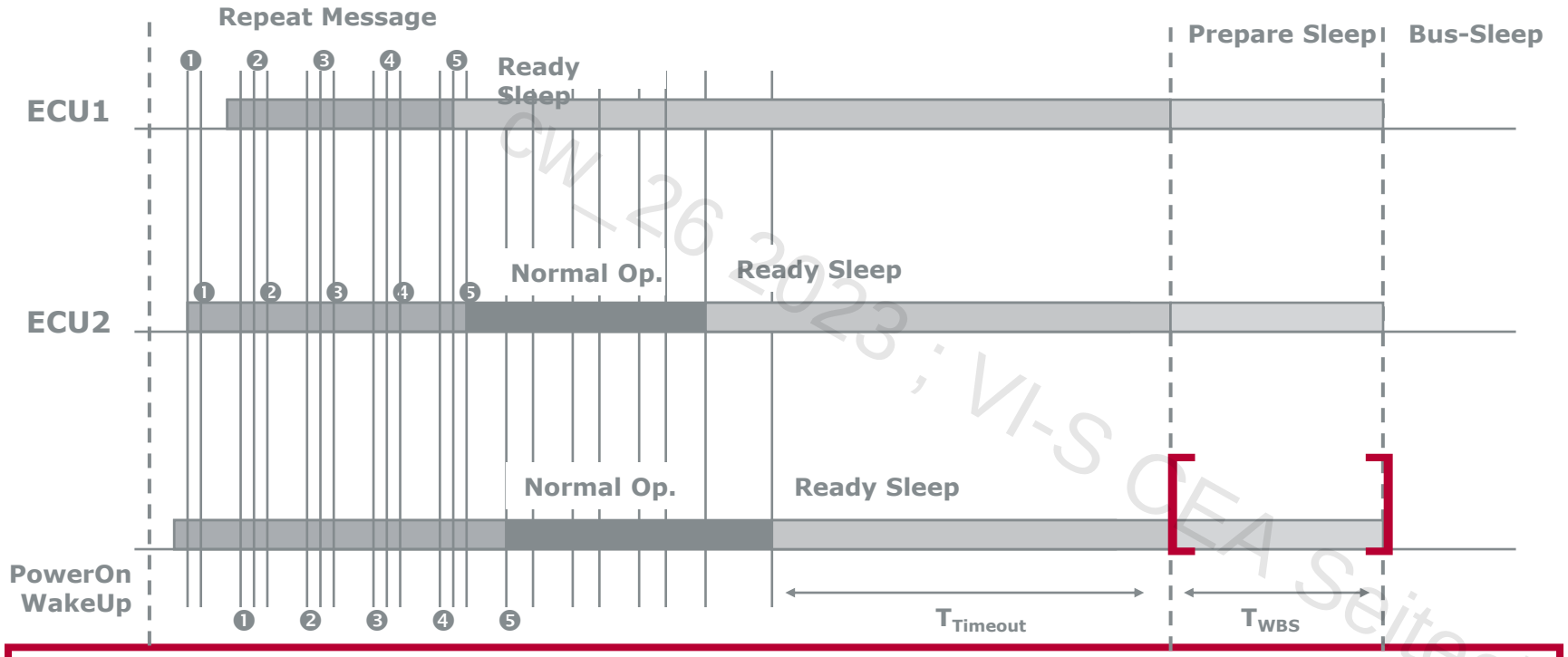
Network Mode (CAN)



- Reception and transmission of application messages enabled
- Repeat Message State
  - cyclic transmission of NM messages for a certain time
  - all NM active nodes participate in NM message communication
- Normal Operation State
  - transmission of NM messages (if activated reduced transmission)
- Ready Sleep State
  - transmission of NM messages is stopped

CW 26 2023 ; VI-S CEA Seitech

Prepare Bus-Sleep Mode (CAN)

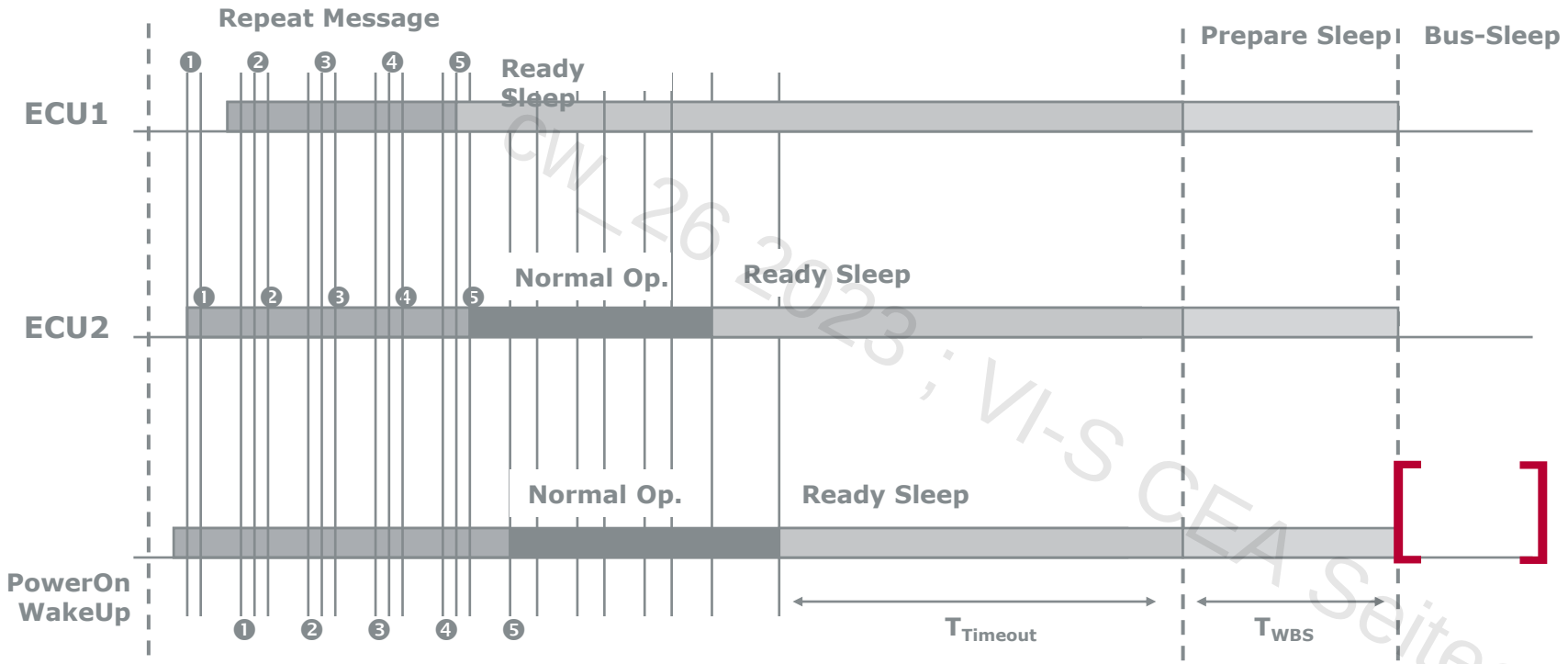


This bus phase corresponds to **SILENT** COMMUNICATION of COMM

- No active participation in the network
- Transmission of application messages is stopped
- Bus calm down period
- Restart of network upon
  - reception of NM message (external)
  - bus communication request (internal)

CW 26 2023 ; VI-S CEA Seitech

Bus-Sleep Mode (CAN)



This bus phase corresponds to **NO** COMMUNICATION of COMM

- Communication bus is shut down
- Restart (wakeup) of network management
  - wake-up on communication bus (external)
  - reception of a NM message (external)
  - application requires bus communication (internal)

CW 26 2023 ; VI-S CEA Seitech

Agenda

- Mode Management
- Wakeup Handling
- BSWM configuration
- ECUM configuration
- Mode Management
- Network Management Algorithm
- ▶ **Mode Manager Concept**
- Service Mapping
- Coming up next

cw_26 2023 ; VI-S CEA Seitech

What is a Mode Manager?

- We can distinguish the following roles
  - Mode Manager, Mode User, Mode Requester
- A **Mode Manager (MMgr)** is an arbitrary SWC or BSW module which has
  - A set of one or more modes which the Mode Manager **provides** as published information to other SWCs (Mode Declaration Group(s))
  - The ability to signalize current mode (state) and mode changes (transitions) over its Port Interface
  - An implementation of the finite state machine corresponding to the published modes
- **Mode User (MUsr)**
  - Arbitrary SWC or the BSWM
  - May **require** / subscribe the transition or state information
- **Mode Requester (MRqr)**
  - Arbitrary SWC or the BSWM with a Mode Request Port

CW_26_2023, VI-S CEA Seitech

What is a Mode Manager?

The **Mode Requester (MRqr)** is very often identical to the **Mode User (MUsr)**, but not always! A Mode Request Port is usually a Sender-Receiver Interface with no special typing. However, such an interface can appear as a mode declaration group when used by the BSWM.

A **Mode Requester** can use the `BswM_RequestMode` API to request a mode from the BSWM in case it uses a **Generic Mode**. If it has its own **Mode Request Interface**, the Mode Request Interface will be used instead.

The **Mode Manager** can either use the `Rte_Switch` API to indicate an internal mode change to the RTE or (in case it uses Generic Modes) it can use a **Generic Mode Trigger Function**.

The **Mode User** can be an SWC which consumes Mode Switch Events or Mode Disabling Dependencies, or it queries the Mode over `Rte_Mode` API. But it can also be a Basic Software module which uses mode information using Mode Trigger Functions.

Important terms for mode management to look up in the AUTOSAR specifications

- Mode Manager
- Mode Declaration Group
- Mode Switch Event
- Mode Disabling Dependency
- Mode Request Port

cw_26_2023 ; VRS CEA Seitech

Elements and mechanisms

- **Mode Declaration Group** (set of supported modes by the MMgr)
  - Defined in the Port Interface of the MMgr
  - Port Interface contains enumerations for the modes
  - Mode Declaration Group has an initial mode value prior to any mode switching
  - Modes are not hierarchically organized (flat view of all modes)
- **Mode Switch Event**
  - Occurs upon transitions between modes (On Mode Exit, On Transition, On Mode Entry)
  - Can be a trigger for a runnable in an arbitrary SWC
  - Can not have a respective waitpoint inside a runnable
- **Mode Disabling Dependency**
  - Any RTE Event can be equipped with a “Mode Disabling Dependency”
  - Is valid during the transition into and the mode itself
  - Prerequisite: consuming Mode User must be connected to MMgr

The RTE can signal

Notification of transitions between modes or the current mode which take place inside a Mode Manager

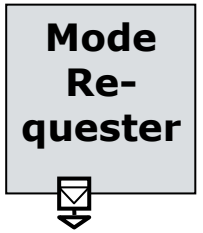
Particularly:

- RTE issues transitions between modes to trigger Runnables (Mode Switch Event(s))

- Prevent a Runnable from being executed during a special mode state (Mode Disabling Dependencies)

CM_26_2023 : VI-SCEA Seitech

Role: Mode Requester

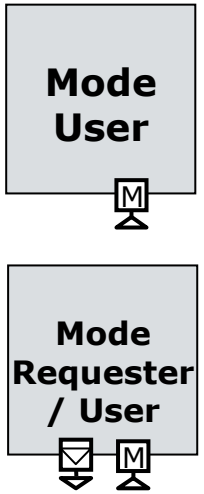


- The **Mode Requester**
  - A SWC can request a mode switch from the MMgr over a **Mode Request P-Port Prototype** (a Sender/Receiver communication)
    - No special annotation in Developer
    - AUTOSAR restriction: isService = TRUE
  - The Mode Requester doesn't need to be an SWC:
    - In case the Mode Requester does not have a Mode Request port, it can use the `BswM_RequestMode()` API (Generic Mode)

CW 36 2023 ; VI-S CEA Seitech

Role: Mode User

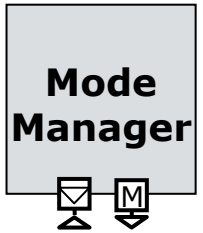
- The **Mode User**
  - The Mode User has a **Mode Switch R-Port Prototype** to be informed about mode changes
  - Can invoke `Rte_Mode` API to get knowledge about a Mode Change
  - A Runnable can be triggered by the Mode Switch
    - On Exit, On Transition, On Entry
  - Frequently, a Mode User is also a **Mode Requester**
    - e.g., with a **Mode Request P-Port Prototype** (a Sender/Receiver communication)



Oct 26 2023 : WS CEA Seitech

Note: In ASR3.x, the Rte_Feedback API could be used for modes by Mode users. In ASR 4.x this is no longer the case. In ASR4.x the Feedback API is dedicated to DataSendCompletion only.

Role: Mode Manager

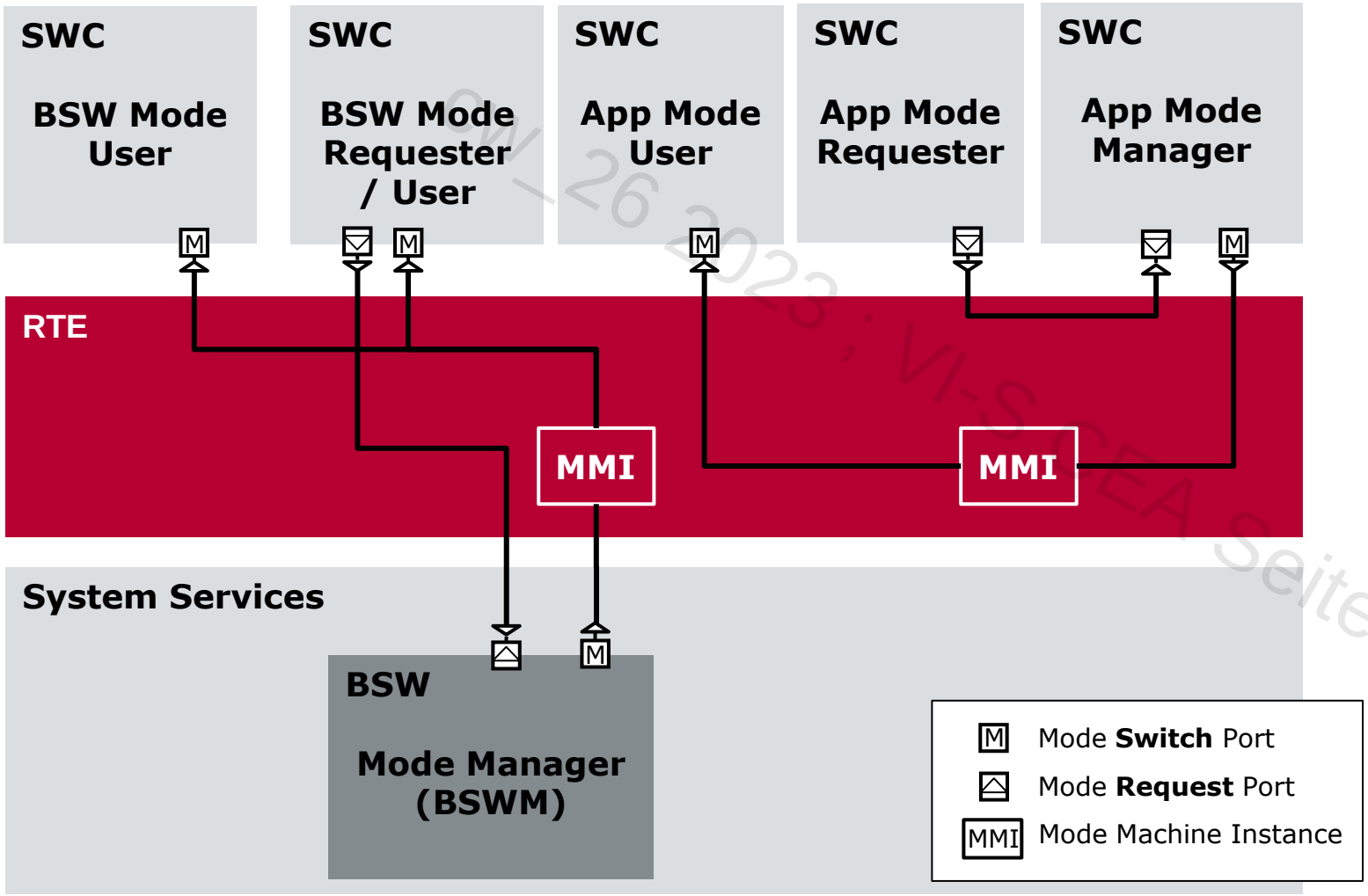


- The **Mode Manager** uses
  - `Rte_Switch` API to indicate the switch into a requested mode to the RTE
    - The RTE keeps a Mode Machine Instance (MMI) to reflect the mode system-wide.
  - `Rte_SwitchAck` API to get a Mode Switch Acknowledge in a non-blocking or blocking way.
  - Additionally, `Rte_Mode` can be called to query the current MMI state in a non-blocking way.
- The RTE indicates the Mode Switch over a **Mode Switch Port** (containing a Mode Declaration Group) to Mode Users as soon as the RTE initiates the mode transition.

cw 28 2023 VI-S CEA Seitech

Of course, the Mode manager can also use the Rte_Mode API to get knowledge about the current MMI state.

Mode Managers vs. Mode Users/Requesters



MMI: Mode Machine Instance. Exists to execute the switching and signaling (Mode Indications) of a ModeDeclarationGroupPrototype between the modes. Each ModeDeclarationGroupPrototype will have one separate MMI inside the RTE.

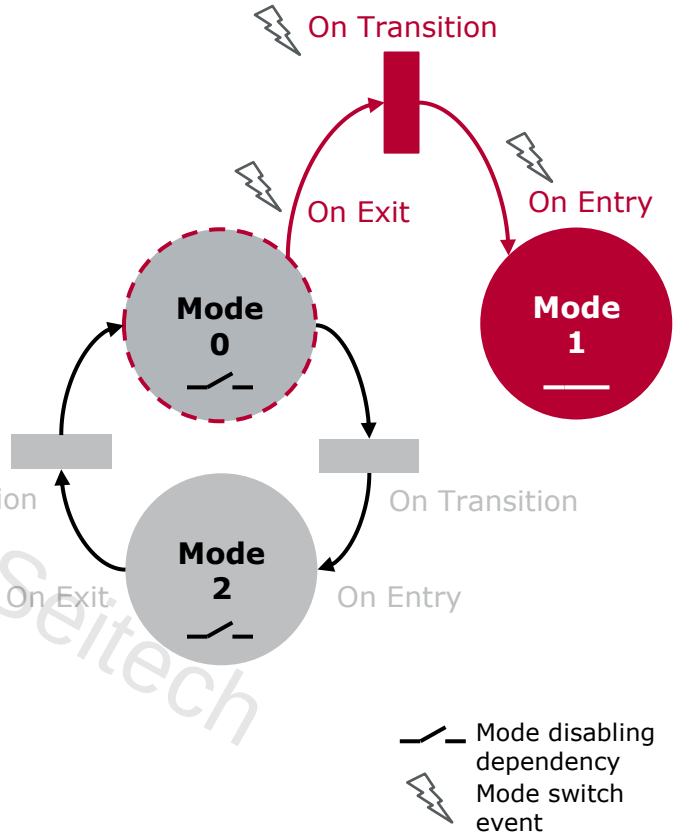
Only the Mode Switch Port contains Mode Declaration Groups; the Mode Request Port does not.

CW 26 2023 ; VI-SCEA Seitenn

Mode Switching

- Mode Switch
  - On Exit
  - On Transition
  - On Entry
- Duration of Mode Disabling
  - From On Exit of last mode until Mode itself
- Mode Switching procedure
  - Synchronous or Asynchronous
  - Defined in
    - *ModeSwitchReceiverComSpec*

cw_26 2023 ; VI-S CEA Seitech



Why does Exit come before Entry? Cleaning up from last mode is necessary prior to switching to a new one!

Mode Switching

The mode switching procedure can be either

- synchronous
- asynchronous

depending on the ability of the Mode Manager's users. If all Mode users are able to use asynchronous operation, the RTE will implement it as such. If at least one of the mode users only supports synchronous operation, the RTE will implement it in the synchronous way. The ability of supporting the mode switch procedure is defined in the ComSepc of the ModeSwitch Receiver (ModeSwitchReceiverComSpec) mode Parameter: supportsAsynchronousModeSwitch.

If the synchronous operation is chosen, there is a waiting condition which will ensure the respective Executable Entities have terminated before entering the next mode.

If the asynchronous operation is chosen, there is no longer a waiting condition for the termination of executable entities. In this case mode disabling-dependent executable entities may not yet have terminated before On Entry or On Exit related-executable entities become active. This can have effects of preemption and delay.

[SWS_Rte_02664] The RTE generator shall reject configurations of a task with

- OnExit ExecutableEntities mapped after OnEntry ExecutableEntities or
- OnTransition ExecutableEntities mapped after OnEntry ExecutableEntities or
- OnExit ExecutableEntities mapped after OnTransition ExecutableEntities

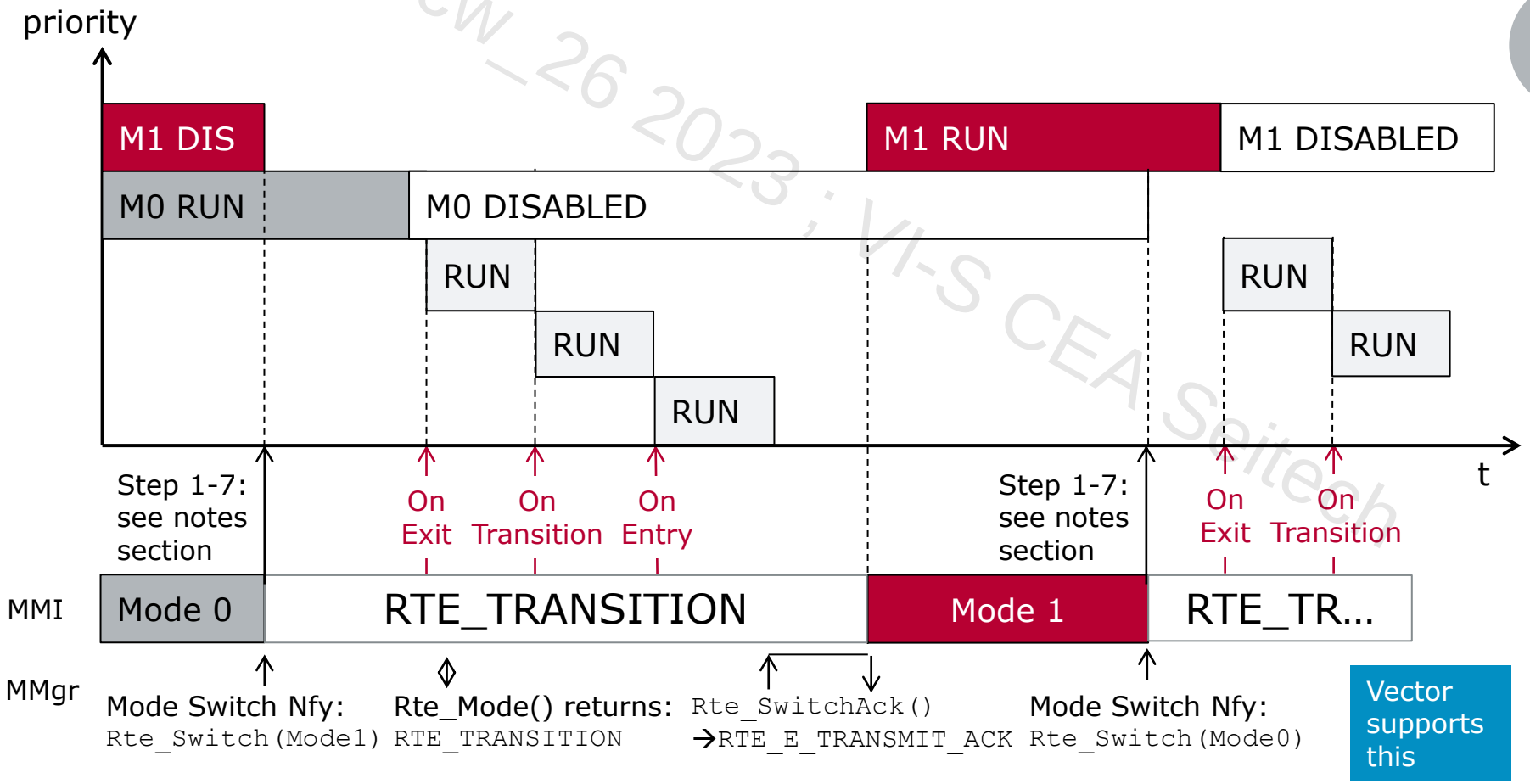
of the same mode machine instance supporting a synchronous mode switching procedure. (SRS_Rte_00143, SRS_Rte_00213, SRS_Rte_00018)

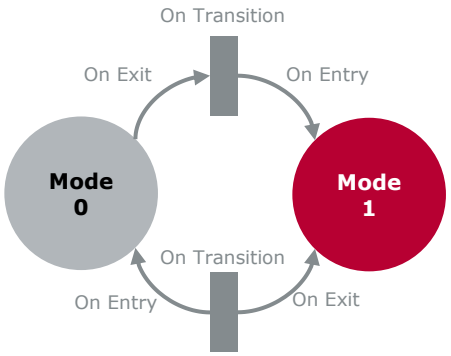
Rationale: This restriction simplifies the implementation of the semantics of asynchronous mode switch.

cw_26 2026 ; VRS OEA Seitech

ModeSwitchReceiverComSpec

- Synchronous mode switching procedure
  - *supportsAsynchronousModeSwitch = FALSE*





cw_26 2023 ; VI-S CEA Saitech

ModeSwitchReceiverComSpec

The Mode manager utilizes the Rte_Switch API to indicate a Mode Change to the RTE.

The RTE then updates the associated MMI accordingly and puts it into the state RTE_TRANSITION.

This state can be queried by the Rte_Mode API in the MMgr. The MMgr can also wait for the mode switch acknowledge by calling Rte_SwitchAck.

If the Mode Switch has been performed successfully, the RTE signalizes RTE_E_TRANSMIT_ACK.

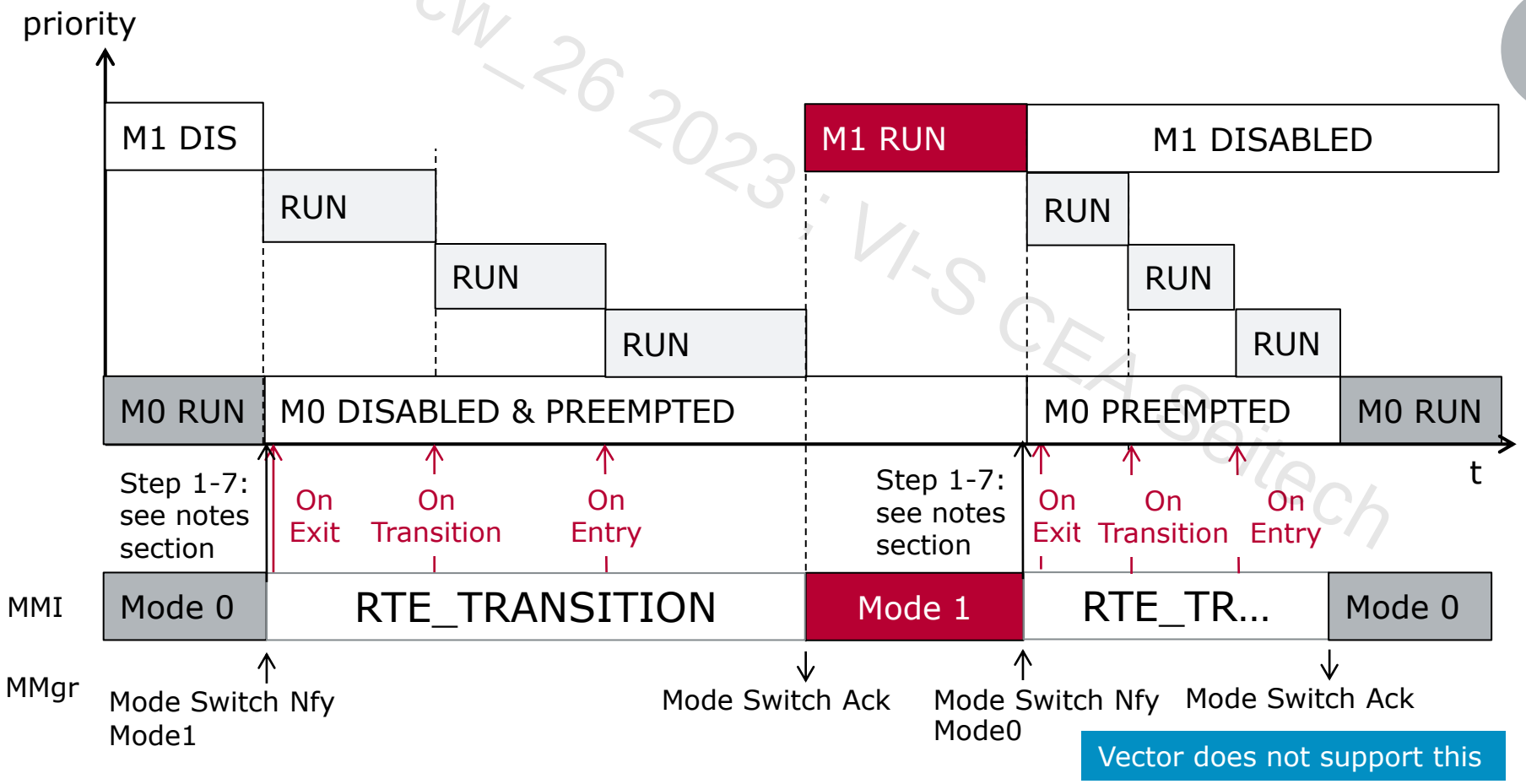
Upon Reception of a mode switch Notification from a MMgr to the RTE, the RTE performs the following steps:

1. Activate mode disablings for the next mode
2. Wait for the newly disabled ExecutableEntities to terminate in case of synchronous mode switching procedure
3. Execute ‘clean up ExecutableEntities’
4. Wait for the ‘clean up ExecutableEntities’ to terminate in case of synchronous mode switching procedure
5. Execute ‘initialization ExecutableEntities’
6. Wait for the ‘initialization ExecutableEntities’ to terminate in case of synchronous mode switching procedure
7. Deactivate mode disablings for the previous modes and enable ExecutableEntities that have been disabled in the previous mode.

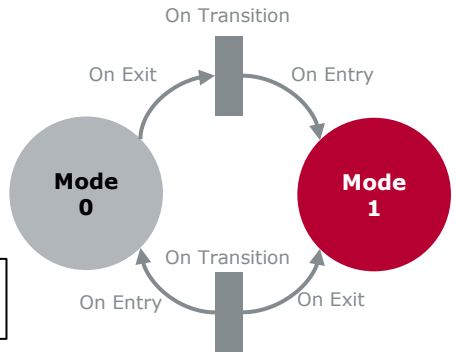
CW_26 2023 ; VI-S CEA Seitech

ModeSwitchReceiverComSpec

- Asynchronous mode switching procedure
  - *supportsAsynchronousModeSwitch = TRUE*



Vector does not support this



cw_26 2023 : VI-S CEA Sertech

ModeSwitchReceiverComSpec

Upon Reception of a mode switch Notification from a MMgr to the RTE, the RTE performs the following steps:

1. Activate mode disablings for the next mode
2. Wait for the newly disabled ExecutableEntities to terminate in case of synchronous mode switching procedure
3. Execute ‘clean up ExecutableEntities’
4. Wait for the ‘clean up ExecutableEntities’ to terminate in case of synchronous mode switching procedure
5. Execute ‘initialization ExecutableEntities’
6. Wait for the ‘initialization ExecutableEntities’ to terminate in case of synchronous mode switching procedure
7. Deactivate mode disablings for the previous modes and enable ExecutableEntities that have been disabled in the previous mode.

cw 26 2023 ; VRS CEA Scitech

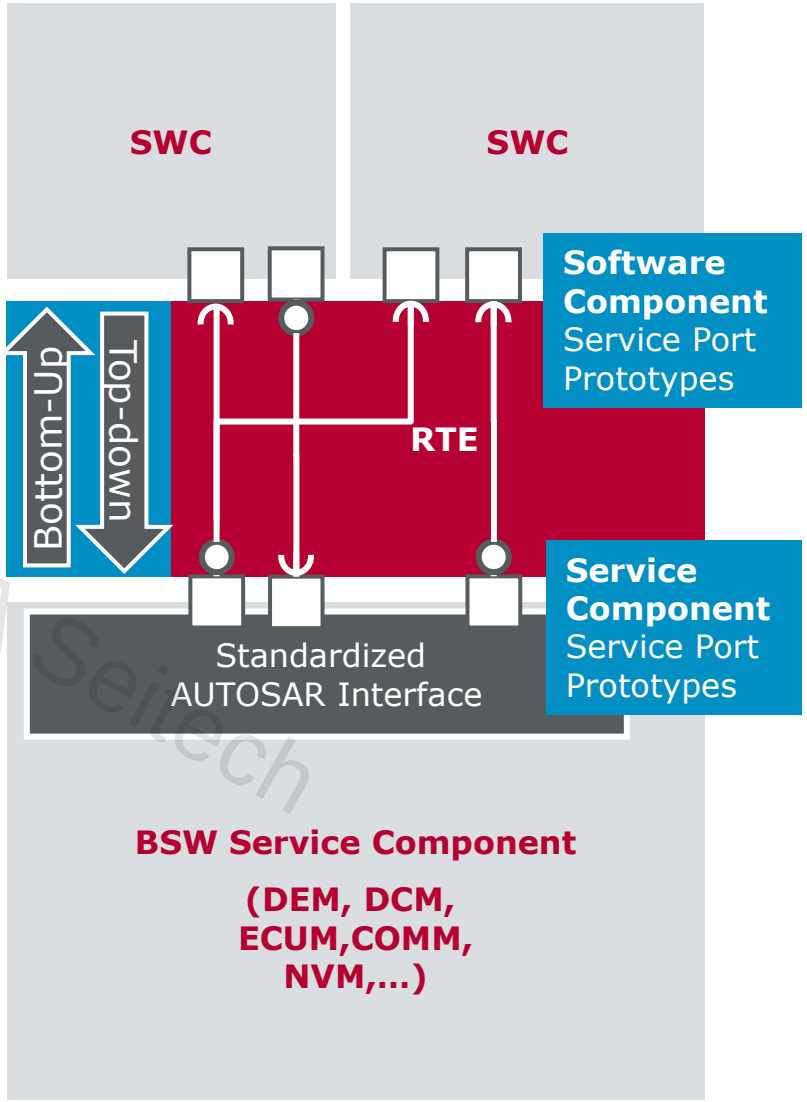
Agenda

- Mode Management
- Wakeup Handling
- BSWM configuration
- ECUM configuration
- Mode Management
- Network Management Algorithm
- Mode Manager Concept
- ▶ **Service Mapping**
- Coming up next

cw_26 2023 ; VI-S CEA Seitech

Service Mapping

- Assigning Service Ports of Software Components to Service Ports of Service Components or vice versa
- Bottom-up service configuration
  1. Configure a service module in the basic software
  2. Assign the respective service port prototypes to the SWCs where appropriate
- Top-down service configuration
  1. Use Service Needs
     > Document at the SWC level what is required from the BSW as a service
  2. Derive a service module configuration based on these requirements
  3. Configure the basic software service module in detail



cw 26 2023, VI-S CEA Seitech

Service Component Configuration (one-stop solution)

Execute Configurator
Configurator Pro

1. configure service component

Configurator Pro

Configure BSW modules with **Standardized AUTOSAR Interface**

→ so-called **Service Components**

ECUC

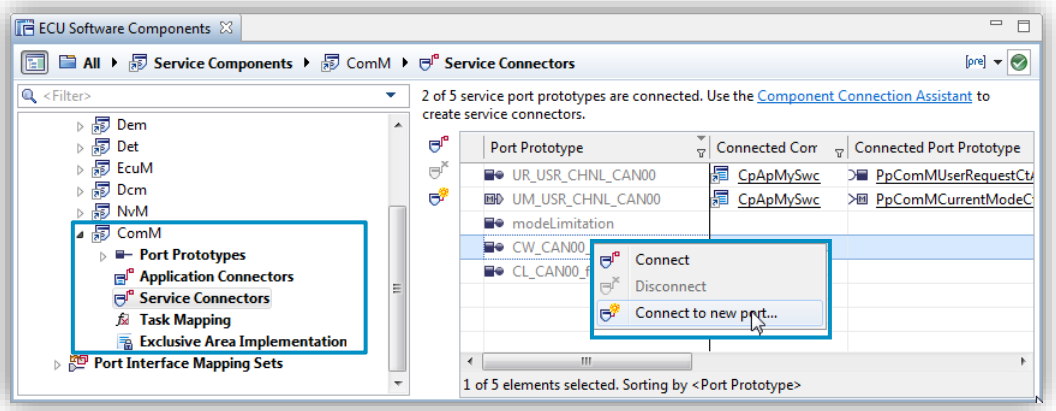
Developer

IMPORT XML file

Functional interface description of the BSW

<BSW>_swc.arxml
e.g. ComM_swc.arxml

Background operation



2. Create Service Component Prototype

3. Equip SWCs with Service Port Prototypes and create the Service mapping and create Server Runnables in one step

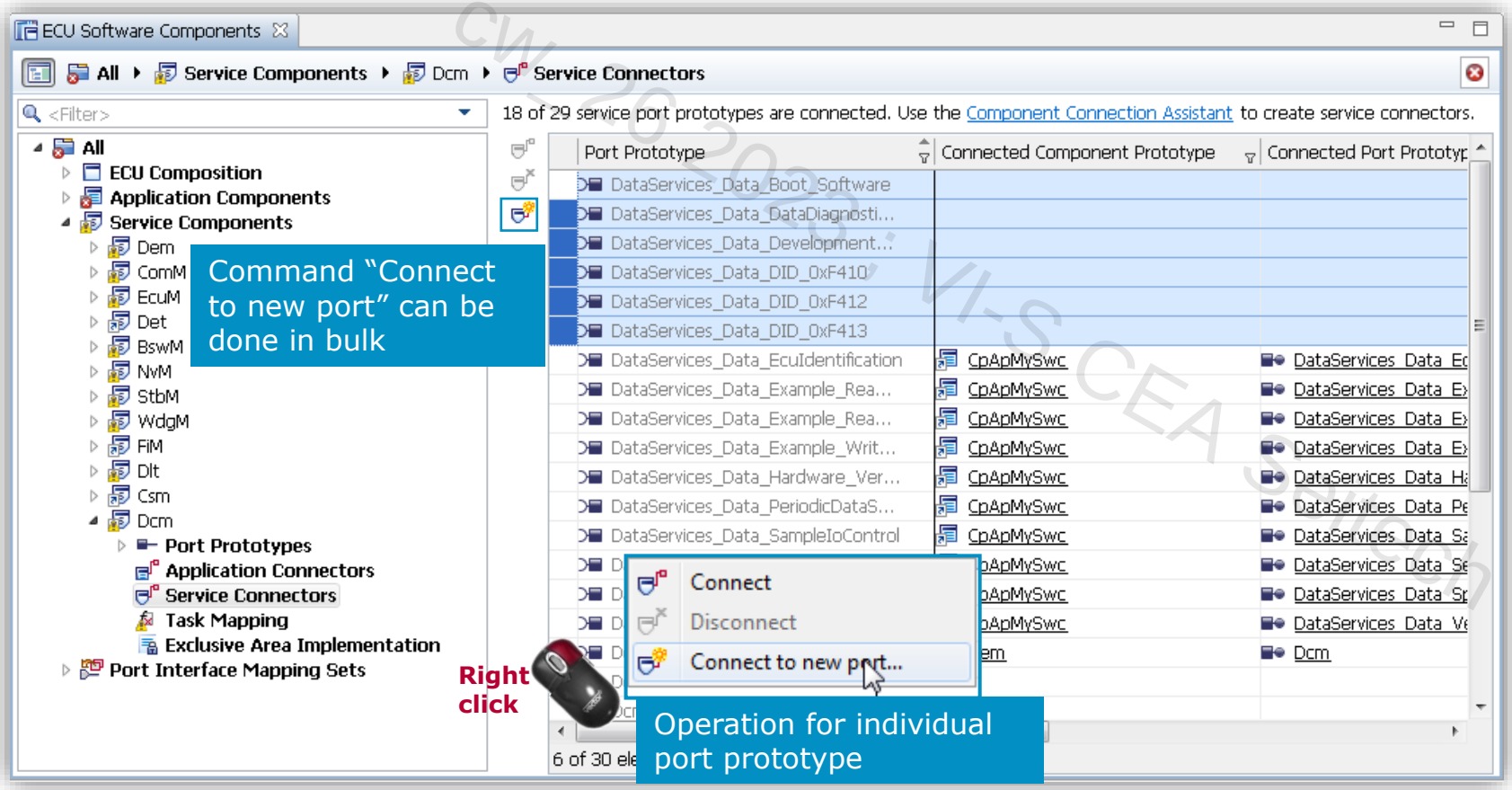
Bottom-Up

In Configurator you can configure the Service Component, create Service Port Prototypes at the SWC and create the Service mapping

CW_26_2023_VTS_CEA Seitech

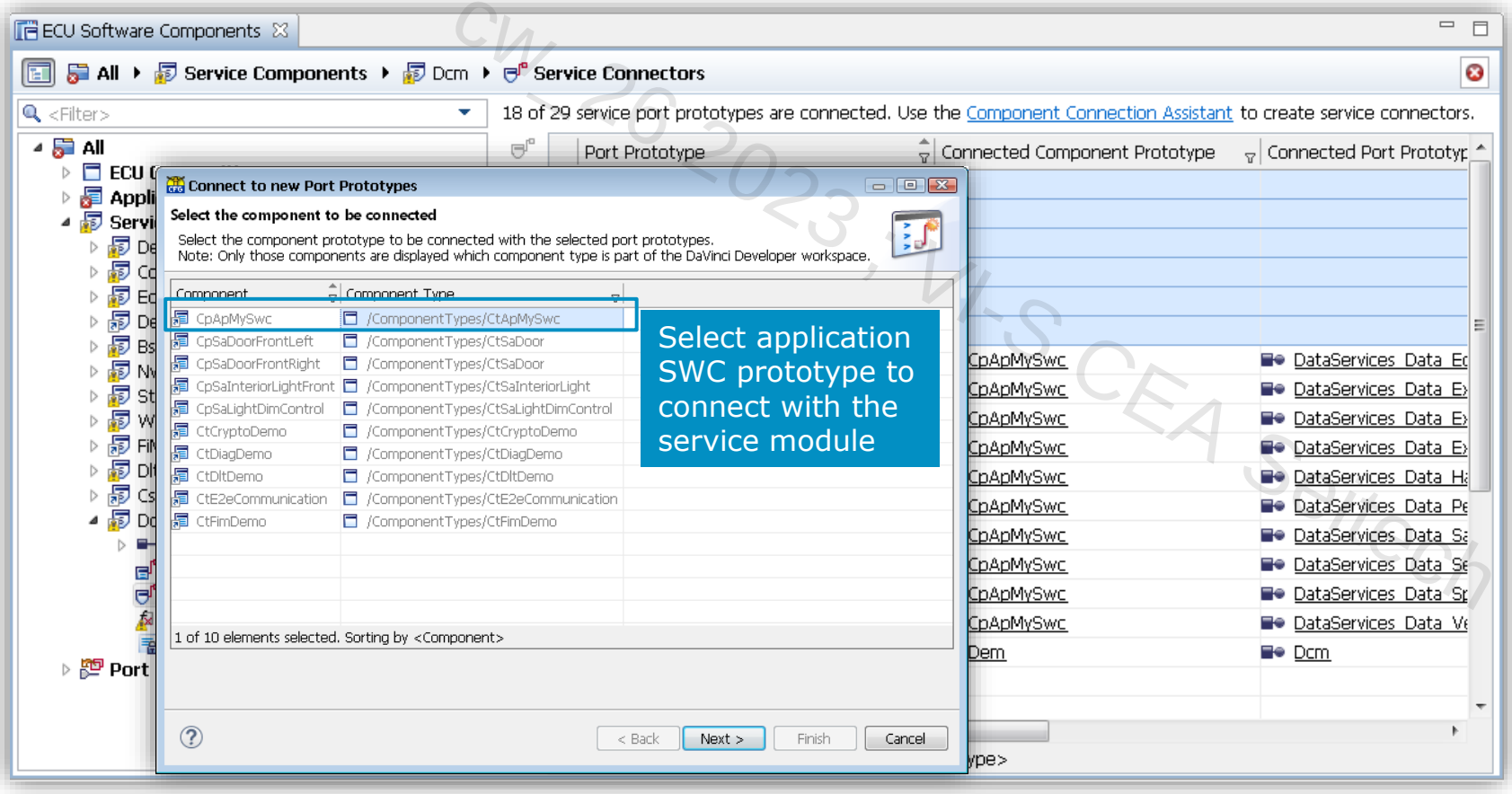
Bottom-up Service Ports (1)

- Select service port prototype to support



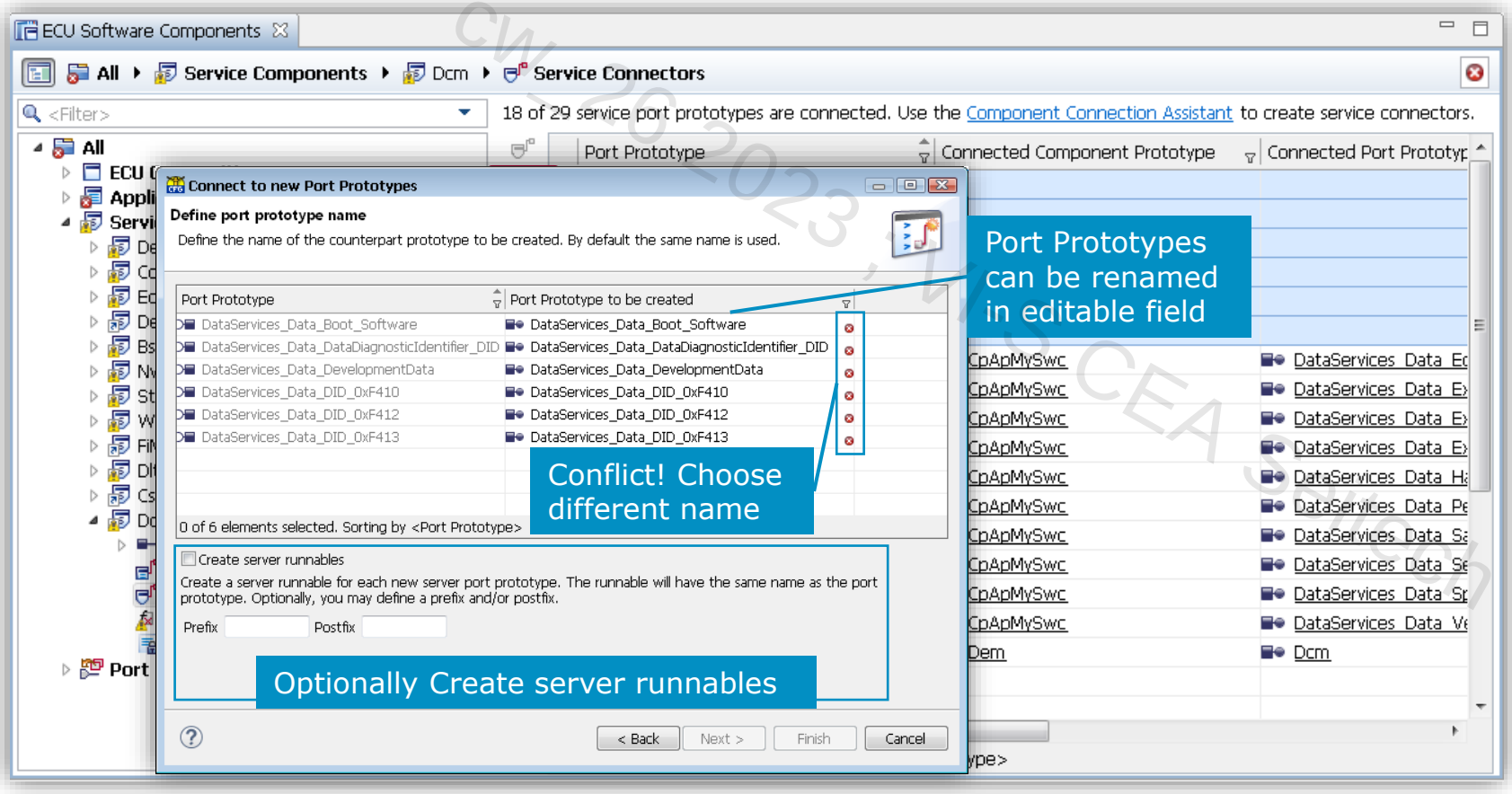
Bottom-up Service Ports (2)

- Choose SWC which shall implement or call the service

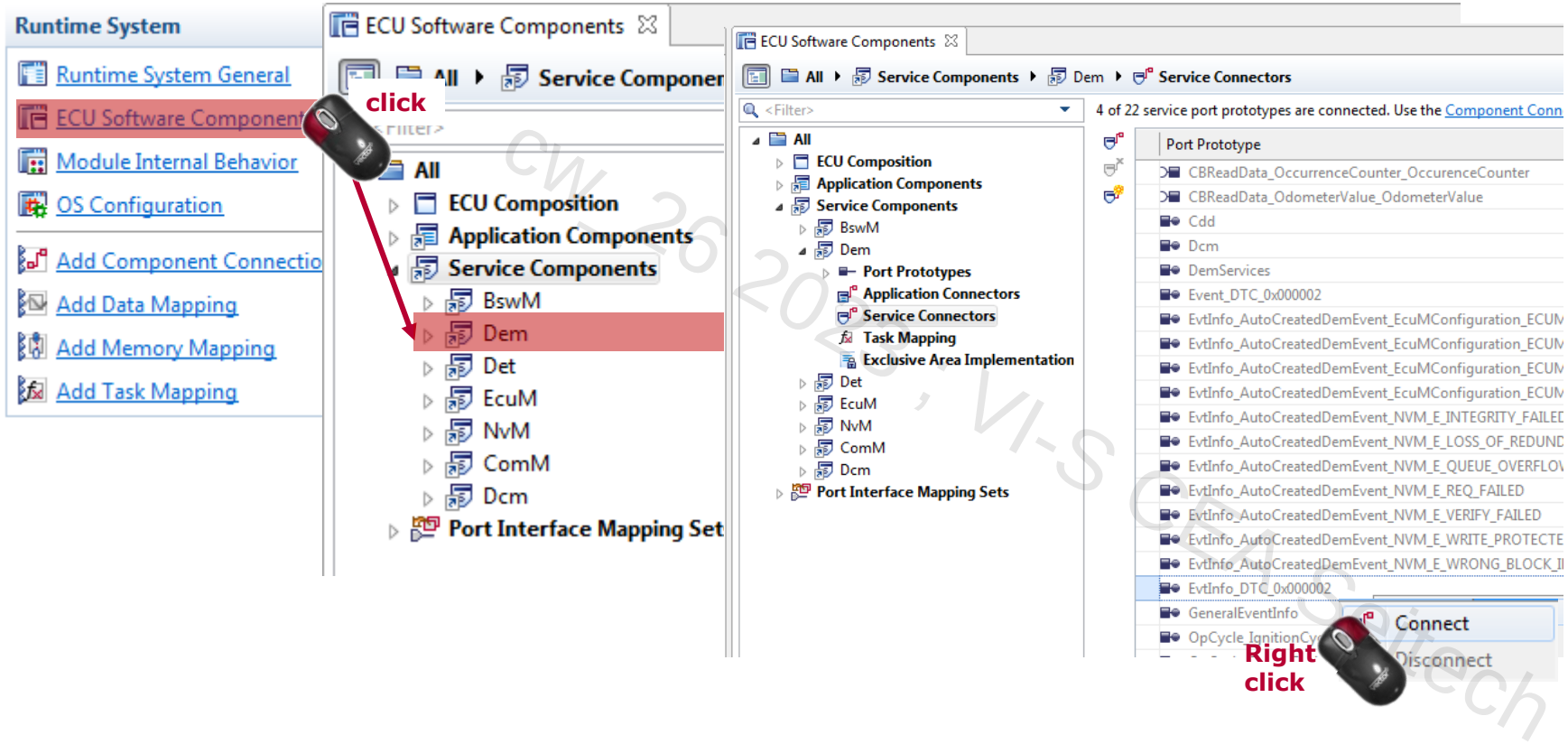


Bottom-up Service Ports (3)

- Create Port Prototypes and in case of Server Ports, create server runnables as well.

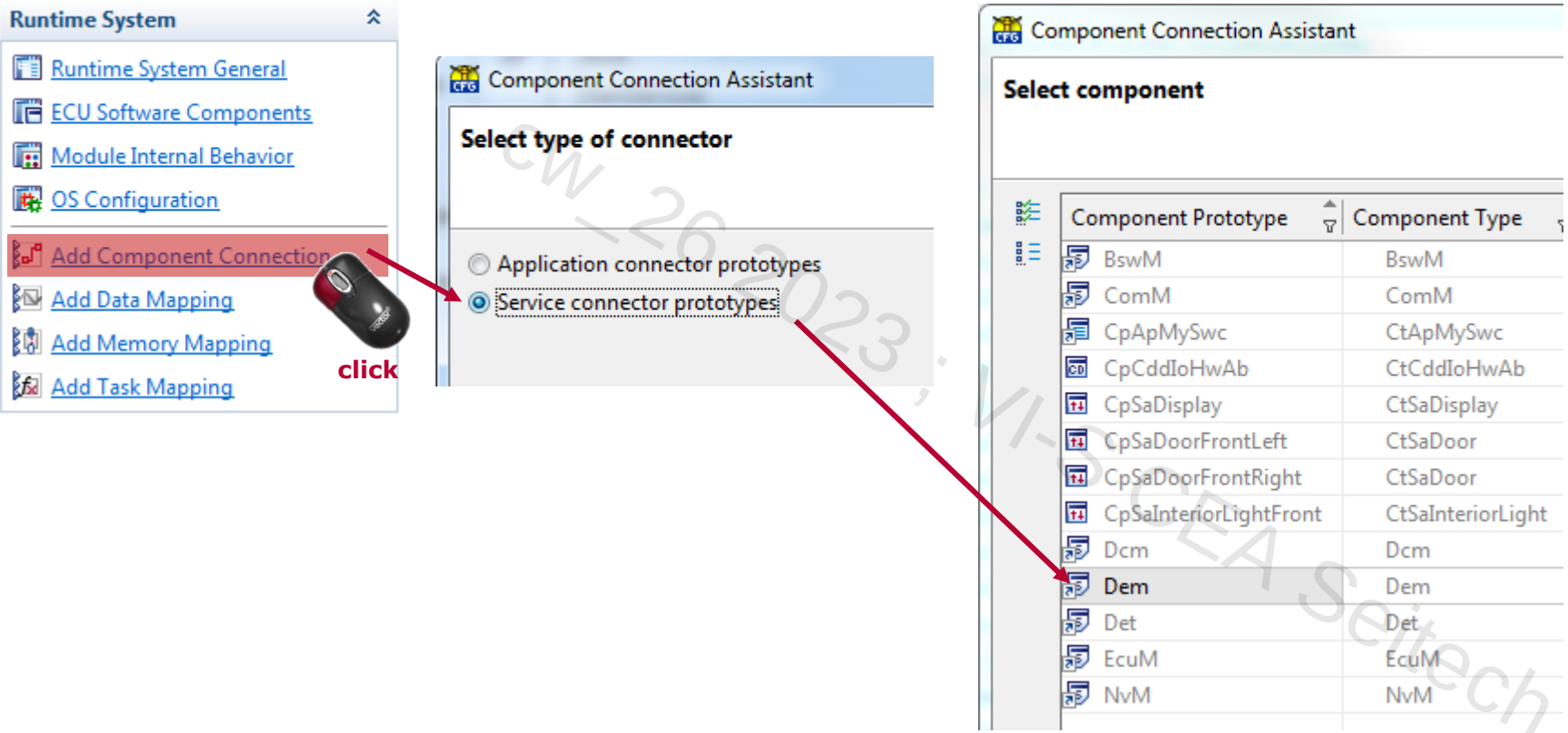


Service Mapping in the ECU Software Components View



- Service mapping is visible in the ECU Software Components View. Here you can also perform service mappings.

Service Mapping in the Component Connection Assistant



- You can also do the service mapping in a global view by using the Component Connection Assistant.

Result of Service Mapping

ECU Software Components

All ▸ Service Components ▸ Dem ▸ Service Connectors

<Filter>

- All
  - ECU Composition
  - Application Components
  - Service Components
    - BswM
    - Dem
      - Port Prototypes
      - Application Connectors
      - Service Connectors
      - Task Mapping
      - Exclusive Area Implementation
    - Det
    - EcuM
    - NvM
    - ComM
    - Dcm
  - Port Interface Mapping Sets

4 of 22 service port prototypes are connected. Use the Component Connection Assistant to create service connectors.

Port Prototype	Connected Component Prototype	Connected Port Prototype
CBReadData_OccurrenceCounter_OccurenceCounter	CpApMySwc	CBReadData_OccurrenceCounter_OccurenceCounter
CBReadData_OdometerValue_OdometerValue	CpApMySwc	CBReadData_OdometerValue_OdometerValue
Cdd		
Dcm		
DemServices		
Event_DTC_0x000002	CpSaInteriorLightFront	Event_DTC_0x000002
EvtInfo_AutoCreatedDemEvent_EcuMConfiguration_ECUM_E_ALL_RUN_REQUESTS_KILLED		
EvtInfo_AutoCreatedDemEvent_EcuMConfiguration_ECUM_E_CONFIGURATION_DATA_INCONSISTENT		
EvtInfo_AutoCreatedDemEvent_EcuMConfiguration_ECUM_E_IMPROPER_CALLER		
EvtInfo_AutoCreatedDemEvent_EcuMConfiguration_ECUM_E_RAM_CHECK_FAILED		
EvtInfo_AutoCreatedDemEvent_NVM_E_INTEGRITY_FAILED		
EvtInfo_AutoCreatedDemEvent_NVM_E_LOSS_OF_REDUNDANCY		
EvtInfo_AutoCreatedDemEvent_NVM_E_QUEUE_OVERFLOW		
EvtInfo_AutoCreatedDemEvent_NVM_E_REQ_FAILED		
EvtInfo_AutoCreatedDemEvent_NVM_E_VERIFY_FAILED		
EvtInfo_AutoCreatedDemEvent_NVM_E_WRITE_PROTECTED		
EvtInfo_AutoCreatedDemEvent_NVM_E_WRONG_BLOCK_ID		
EvtInfo_DTC_0x000002		
GeneralEventInfo		
OpCycle_IgnitionCycle	CpApMySwc	OpCycle_IgnitionCycle
OpCycle_PowerCycle		
OverflowIndPrimaryMemory		

ow – 26.2023 ; VI-S CEA Seitech

Mode Management

Exercise: Mode Management

cw_26 2023 - VI-S CEA Seitech